



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea in Ingegneria delle Tecnologie per l'Impresa Digitale (ITID)-da
tenere ITID?

Tesi di Laurea Triennale

Monitoraggio, manutenzione e sicurezza per smart city: dalla gestione manuale ad un'architettura automatizzata basata su Zabbix

Relatore:

Prof. Francesco Gringoli

Laureando:

Brando Calderara

Matricola n. 746352

Correlatore Aziendale:

Nome Correlatore Secoval, da inserire?

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare, della durata di circa duecentoventicinque ore, dal laureando Brando Calderara presso l'azienda Secoval Srl. Il progetto ha avuto come obiettivo principale la realizzazione di un sistema di monitoraggio centralizzato e automatizzato per le infrastrutture Smart City del territorio, superando i limiti dei precedenti sistemi manuali e statici. Lo scopo è stato sviluppare e integrare un sistema di monitoraggio basato su Zabbix...

Il progetto di stage ha avuto come obiettivo principale la realizzazione del backend e delle API per una web application di e-commerce. Lo scopo è stato sviluppare un sistema robusto e scalabile, in grado di gestire in maniera efficace le operazioni di un negozio online. L'applicativo non si limita infatti ad un tradizionale e-commerce, con prodotti e carrello, ma introduce dinamiche personalizzate richieste dall'azienda per adattare le modalità di vendita e l'esperienza utente alla piattaforma. Il mio compito principale è stato la progettazione e l'implementazione del backend: dalla definizione del database, realizzato con PostgreSQL, allo sviluppo dell'applicazione server mediante il framework Django, con particolare attenzione alla creazione di API RESTful per garantire una comunicazione efficiente tra frontend e backend. In particolare, mi sono occupato dello sviluppo delle sezioni relative ai profili utente, con la gestione delle diverse tipologie previste, e del sistema di autenticazione tramite token JWT, comprendente le principali operazioni di login e registrazione. Ho inoltre sviluppato la gestione del catalogo prodotti, integrando le dinamiche personalizzate richieste, nonché i sistemi di filtraggio e ricerca. Per ciascuna di queste sezioni è stata implementata anche un'interfaccia di amministrazione, destinata al personale dell'azienda per il monitoraggio e la gestione dei dati. Infine, era prevista l'implementazione di test per le componenti sviluppate. Il lavoro svolto durante lo stage ha richiesto una costante collaborazione con il team, al fine di garantire che il backend fosse coerente con le esigenze progettuali e si integrasse senza problemi con il frontend [Qui aggiungeremo 10- 15 righe di riassunto del progetto: Zabbix, Python, API, Interacta].

Indice

Sommario	1
Elenco delle figure	4
Elenco delle tabelle	5
1 Introduzione	6
1.1 L'Azienda: Secoval S.r.l.	6
1.2 Il Progetto di Stage	7
1.3 Obiettivi e Analisi dei Requisiti	8
2 Tecnologie Utilizzate	10
2.1 Zabbix 7.0 LTS	10
2.2 Python, Venv e i package pip utilizzati	11
2.3 Scripting Bash e Ambiente Linux (Cron, Logrotate)	12
2.4 Interacta e JSON Web Token (JWT)	13
3 Architettura e Configuration Management	15
3.1 I CMDB aziendali (Excel) come Source of Truth	15
3.2 Sviluppo del motore di Provisioning in Python via Zabbix API e setup della Macchina Virtuale Ubuntu	17
3.3 Integrazione GIS: Estrazione coordinate tramite Excel e visualizzazione in Zabbix	19
3.4 Organizzazione gerarchica in Host Groups degli host e i codici comunali	21
4 Implementazione: Controlli Attivi ed Eventi TVCC	24
4.1 Logiche di Polling vs Trapping in Zabbix	24
4.2 Sviluppo del Middleware e Ascolto degli Stream Hikvision ISAPI	26
4.3 Inoltro asincrono tramite Zabbix Sender	29
4.4 Definizione di Trigger e Discovery Actions OPPURE MEGLIO Configurazione Pratica di Trigger e Actions per gli Eventi TVCC	30
5 Integrazione con il Sistema di Ticketing Interacta	32
5.1 Sicurezza e Autenticazione Server-to-Server	32
5.2 Generazione e firma del token JWT (Algoritmo RS512)	34
5.3 Sviluppo del Custom AlertScript in Zabbix	36
6 Analisi Dati, KPI e Manutenzione di Sistema	39
6.1 Esportazione degli eventi in formato strutturato (CSV)	39
6.2 Sviluppo del motore Python per il calcolo dei KPI	41

6.3	Automazione reportistica tramite Crontab e SMTP	43
6.4	Gestione dei log e manutenzione (Logrotate)	44
7	Conclusioni	47
7.1	Raggiungimento degli Obiettivi	47
7.2	Limiti del sistema e Sviluppi Futuri	48
7.3	Valutazione Personale	50
	Glossario	51
7.4	Terminologia	51
	Bibliografia	60
	Ringraziamenti	62

Elenco delle figure

2.1	Logo Zabbix	10
2.2	Logo Python	11
2.3	Logo Interacta	13

Elenco delle tabelle

Capitolo 1

Introduzione

1.1 L'Azienda: Secoval S.r.l.

Secoval srl è una società a capitale interamente pubblico, partecipata dalla Comunità Montana di Valle Sabbia e da oltre 40 enti locali. Opera come supporto tecnologico e operativo per le Pubbliche Amministrazioni del nord-est bresciano, con sede a Vestone. Il suo ruolo è supportare la trasformazione digitale dei Comuni, compresi quelli più periferici, fornendo servizi amministrativi, informatici e gestionali. (Secoval S.r.l., 2026)

Le principali aree di operatività dell'azienda includono:

- **Infrastrutture IT e cloud:** gestione di data center, servizi cloud, sicurezza informatica e interoperabilità per gli enti pubblici.
- **Sistemi informativi territoriali:** sviluppo di reti (inclusa la fibra ottica), digitalizzazione del catasto e cartografia.
- **Servizi sovracomunali e gestionali:** gestione dei tributi locali, supporto amministrativo e al personale degli enti.

Nel contesto tecnologico delle Smart City, Secoval gestisce apparati tra cui telecamere per la videosorveglianza urbana (TVCC), Access Point (AP) per la connettività pubblica, switch industriali e gateway LTE per le aree non cablate. Essi fanno parte di reti distribuite sul territorio per l'erogazione di servizi diretti ai cittadini.

Il progetto svolto durante il tirocinio si inserisce direttamente in questo scenario operativo, con l'obiettivo di superare la gestione manuale implementando un'architettura automatizzata e centralizzata basata su Zabbix, in modo che si realizzi un sistema di monitoraggio solido per la rete di apparati di Secoval, abbastanza vasta e appena installata quindi instabile. Il lavoro ha previsto l'automazione del discovery e dell'assegnazione dei metadati degli apparati tramite fogli Excel, l'integrazione con i flussi di allarme nativi dei dispositivi periferici (come gli eventi ISAPI Hikvision) e la comunicazione sincrona e crittografata con la piattaforma cloud di ticketing aziendale Interacta per la gestione asincrona dei guasti da parte degli operatori.

1.2 Il Progetto di Stage

[Descrizione ad alto livello del progetto: l'esigenza di monitorare TVCC, AP, Switch e LTE. Integrazione con script esterni per implementare funzionalità e per fare analisi dei dati. Integrazione con Interacta].

Il progetto di stage ha come obiettivo la progettazione e l'implementazione di un'architettura di monitoraggio centralizzata basata su Zabbix 7.0, destinata alla gestione dell'infrastruttura di rete Smart City di Secoval. L'esigenza primaria è superare i limiti operativi dei sistemi di monitoraggio pregressi (come PRTG), caratterizzati da un'elevata gestione manuale e da una scarsa scalabilità nell'inserimento di nuovi nodi.

L'infrastruttura oggetto di monitoraggio è altamente eterogenea e distribuita capillarmente sul territorio. I dispositivi principali comprendono:

- **Telecamere di videosorveglianza (TVCC):** principalmente apparati Hikvision, che richiedono il controllo di raggiungibilità e dell'integrità del flusso video.
- **Access Point (AP):** nodi per l'erogazione di connettività Wi-Fi pubblica nei centri urbani.
- **Switch di distribuzione:** apparati industriali per l'aggregazione del traffico periferico.
- **Dispositivi LTE/5G:** gateway per l'uplink in aree remote non coperte da fibra ottica.

Per garantire una gestione scalabile, il progetto fa largo uso di script esterni (Python e Bash) per espandere le funzionalità native del sistema. Il tradizionale processo di *Network Discovery* basato su ICMP è stato sostituito da un meccanismo di provisioning dinamico via Zabbix API: uno script Python legge un CMDB aziendale in formato Excel e popola automaticamente l'inventario degli host associando l'IP a metadati specifici come l'ID Host, l'ID Palo e le coordinate geografiche. Sul fronte degli eventi, per le telecamere Hikvision è stato sviluppato un middleware Python in ascolto sugli stream ISAPI, incaricato di intercettare alert critici (es. *Video Tampering*) e inoltrarli asincronamente tramite protocollo Zabbix Sender.

Un requisito fondamentale del progetto è l'integrazione architetturale con **Interacta**, la piattaforma cloud aziendale per il ticketing. La comunicazione bidirezionale è gestita da un Custom AlertScript eseguito localmente sul server Zabbix, il quale apre e risolve automaticamente i ticket di intervento al variare dello stato dei trigger. La sicurezza di questa comunicazione Server-to-Server è garantita dall'utilizzo di JSON Web Token (JWT) generati e firmati crittograficamente con algoritmo asimmetrico RSA (RS512) tramite librerie Python dedicate.

A completamento dell'infrastruttura, sono stati integrati strumenti per l'analisi dei dati operativi. Le serie storiche degli allarmi vengono esportate su file system in formato testuale (CSV) ed elaborate da un motore di analisi Python per estrarre Key Performance Indicators (KPI), tra cui il *Mean Time To Resolution* (MTTR) e l'incidenza di anomalie hardware (flapping). L'orchestrazione di questi flussi di esportazione, l'invio della reportistica e la rotazione sicura dei file di log avvengono a livello di sistema operativo Linux sfruttando utilità native come Crontab e Logrotate.

1.3 Obiettivi e Analisi dei Requisiti

[Classificazione dei requisiti funzionali e di vincolo. Cosa ti ha chiesto di fare Secoval? Quali erano le limitazioni?].

"Attività oggetto del tirocinio

Il progetto formativo mira a trasformare l'infrastruttura di rete (videosorveglianza, Wi-Fi pubblico) che serve oltre 30 Comuni in un sistema Smart City integrato lavorando su tre pilastri:

Sviluppo e miglioramento del monitoraggio di rete.

Integrazione di tutti gli apparati (telecamere, switch, AP) con la Cartografia GIS per creare uno strumento di gestione georeferenziato.

Cybersecurity e Compliance GDPR, verificando configurazioni di firewall, captive portal, descrivendo le procedure operative per la gestione degli account per garantire la sicurezza dell'intera rete."

Il progetto formativo concordato con Secoval S.r.l. si inserisce in un piano di modernizzazione dell'infrastruttura di rete che serve oltre 30 Comuni. L'esigenza primaria è trasformare la rete periferica esistente, composta da dispositivi di videosorveglianza urbana e access point Wi-Fi, in un ecosistema Smart City integrato, reattivo e facilmente scalabile.

Come definito dal patto formativo, il lavoro si articola su tre pilastri fondamentali:

1. **Sviluppo e miglioramento del monitoraggio di rete:** superare i limiti dell'inserimento manuale e del tradizionale discovery basato su ICMP, implementando un'architettura centralizzata su Zabbix 7.0 capace di auto-configurarsi e orchestrare dinamicamente centinaia di nodi eterogenei.
2. **Integrazione con la Cartografia GIS:** collegare gli apparati hardware (telecamere, switch, AP) al territorio. Questo prevede lo sviluppo di script per l'estrazione automatica di coordinate spaziali e metadati da file Excel e PDF, al fine di popolare l'Host Inventory di Zabbix e creare uno strumento di gestione visuale e georeferenziato.
3. **Cybersecurity e Compliance GDPR:** garantire la sicurezza delle comunicazioni machine-to-machine, verificare le configurazioni degli apparati per rispettare gli standard normativi e descrivere procedure operative rigorose per la gestione degli account e il logging degli eventi.

Classificazione dei Requisiti

L'analisi del dominio applicativo e dei vincoli imposti dall'infrastruttura aziendale ha portato alla definizione dei seguenti requisiti tecnici e funzionali.

Requisiti Funzionali

- **Provisioning dinamico tramite CMDB:** il sistema deve importare gli host leggendo un Configuration Management Database in formato Excel, associando automaticamente gli indirizzi IP a metadati specifici (ID Host, ID Palo).
- **Gestione proattiva degli eventi TVCC:** il monitoraggio deve intercettare logiche applicative complesse, come il *Video Tampering* e il *Motion Detec-*

tion, mediante l'ascolto continuo degli stream ISAPI generati nativamente dalle telecamere Hikvision.

- **Automazione del ticketing (Interacta):** al verificarsi di un disservizio o al suo rientro, il sistema deve interfacciarsi con le API REST del cloud aziendale Interacta per gestire l'apertura e la risoluzione automatica dei ticket di manutenzione.
- **Elaborazione KPI:** il sistema deve estrarre serie storiche in formato CSV per calcolare metriche operative, come il *Mean Time To Resolution* (MT-TR) e l'identificazione di host instabili (Flapping).

Requisiti di Vincolo (Tecnologici e Operativi)

- **Stack Tecnologico:** obbligo di impiegare Zabbix 7.0 LTS come core engine. Le automazioni e le integrazioni di terze parti devono essere sviluppate in linguaggi compatibili con l'ambiente Linux server, primariamente Python 3 e script Bash.
- **Sicurezza crittografica (JWT):** l'integrazione con Interacta impone una rigorosa autenticazione Server-to-Server. Le richieste HTTP devono essere protette tramite JSON Web Token (JWT) compilato in locale e firmato crittograficamente con chiave asimmetrica RSA e algoritmo RS512.
- **Gestione risorse e asincronia:** l'esecuzione di script esterni non deve gravare sui processi di polling di Zabbix. L'interazione deve sfruttare logiche di inoltro asincrono (Zabbix Sender). Inoltre, per evitare l'esaurimento dello spazio su disco, l'output dei processi custom deve essere gestito rigidamente a livello di sistema operativo tramite l'utility **Logrotate**.

Capitolo 2

Tecnologie Utilizzate

[In questo capitolo descriveremo formalmente gli strumenti, includendo i relativi loghi come figure, esattamente come nella tesi di riferimento].

2.1 Zabbix 7.0 LTS

Figura 2.1: Logo Zabbix

Zabbix è una piattaforma open-source di livello enterprise progettata per il monitoraggio distribuito e in tempo reale di metriche di rete, apparati hardware, server e servizi applicativi. L'architettura del sistema si fonda su un modello client-server altamente scalabile, composto dai seguenti moduli principali:

- **Zabbix Server:** il motore centrale sviluppato in linguaggio C. Si occupa della raccolta dei dati, della valutazione logica dei *Trigger* (soglie di allarme) e dell'orchestrazione delle *Actions* (notifiche o script di remediation).
- **Database Relazionale:** il livello di persistenza, tipicamente basato su PostgreSQL o MySQL, destinato all'archiviazione delle configurazioni, degli eventi e delle serie temporali (history e trend).
- **Web Frontend:** l'interfaccia utente sviluppata in PHP, utilizzata per l'amministrazione del sistema, la configurazione visuale e la generazione di dashboard analitiche.
- **Zabbix Proxy e Agent:** demoni impiegati per delegare la raccolta dati in scenari multi-tenant o reti segmentate, limitando il carico sul server centrale e riducendo l'apertura delle porte firewall.

Il funzionamento della piattaforma si basa su due paradigmi complementari di acquisizione dati. Il primo è il *polling* (controllo attivo), in cui il server interroga ciclicamente i dispositivi target tramite protocolli standard come ICMP, SNMP o interrogazioni HTTP/API. Il secondo è il *trapping* (controllo passivo o asincrono), che consente a sistemi esterni di spingere proattivamente i dati verso il server Zabbix. Questo secondo meccanismo si avvale dell'utility a riga di comando **Zabbix Sender** e risulta fondamentale per abbattere i tempi di latenza nella rilevazione degli allarmi.

Per questo progetto è stata adottata la versione 7.0 LTS (Long Term Support). La scelta di una release LTS garantisce stabilità e supporto esteso nel tempo, un requisito imprescindibile per le infrastrutture critiche della Pubblica Amministrazione. Dal punto di vista tecnico, questa versione introduce o consolida funzionalità determinanti per le attività del tirocinio:

- **API JSON-RPC:** un'interfaccia di programmazione strutturata che consente il controllo totale della piattaforma via codice. È stata utilizzata intensivamente per sviluppare il motore Python di provisioning automatico, capace di popolare l'Host Inventory leggendo i metadati dal CMDB aziendale.
- **Custom AlertScripts:** la capacità di lanciare eseguibili esterni (Python, Bash) al verificarsi di un evento. Questo modulo è stato impiegato per integrare Zabbix con la piattaforma cloud Interacta, gestendo l'apertura e la risoluzione dei ticket tramite chiamate crittografate.
- **Data Preprocessing avanzato:** strumenti nativi per il parsing di stringhe (regex), JSONPath e XMLXPath prima della memorizzazione a database. Tali logiche sono essenziali per normalizzare gli eventi asincroni intercettati dai flussi ISAPI delle telecamere Hikvision.

2.2 Python, Venv e i package pip utilizzati

Figura 2.2: Logo Python

Python è un linguaggio di programmazione ad alto livello, interpretato, tipizzato dinamicamente e orientato agli oggetti. La sua sintassi chiara e la vasta disponibilità di librerie lo rendono lo standard di fatto per lo sviluppo di script di automazione, middleware di integrazione e analisi dati. Nel contesto di questo progetto, Python è stato il linguaggio principale utilizzato per estendere le funzionalità di Zabbix e interfacciare i vari sistemi eterogenei (Hikvision, Interacta, CMDB aziendale).

L'esecuzione degli script personalizzati all'interno del server Linux di produzione ha reso necessario l'utilizzo di **venv** (Virtual Environment). Questo strumento nativo di Python permette di creare ambienti virtuali isolati per ogni progetto. L'adozione di **venv** garantisce che le librerie installate per le automazioni di Zabbix non interferiscano con i pacchetti globali del sistema operativo, prevenendo problemi di dipendenze (dependency hell), aumentando la sicurezza e facilitando il porting del codice.

La gestione dei pacchetti aggiuntivi è stata effettuata tramite **pip** (Package Installer for Python). Di seguito si riportano le librerie fondamentali impiegate per lo sviluppo dei moduli del tirocinio:

- **requests:** libreria progettata per semplificare l'esecuzione di richieste HTTP. È stata il motore di comunicazione principale del progetto, utilizzata per interrogare le API JSON-RPC di Zabbix, per inviare i payload all'API REST di Interacta e per mantenere connessioni persistenti (streaming) verso gli endpoint ISAPI delle telecamere Hikvision.

- **pandas (e openpyxl)**: libreria di riferimento per l'analisi e la manipolazione dei dati. È stata impiegata per leggere, filtrare e normalizzare i Configuration Management Database (CMDB) aziendali in formato Excel (`.xlsx`), estrapolando i metadati necessari (IP, ID Host, ID Palo) per il provisioning dinamico degli apparati.
- **pdfplumber**: modulo avanzato per l'estrazione di testo e tabelle da documenti PDF. In combinazione con la libreria nativa **re** (Regular Expressions), ha permesso di effettuare il parsing di relazioni tecniche aziendali per recuperare automaticamente le coordinate geografiche (latitudine e longitudine) e integrarle nell'inventario GIS di Zabbix.
- **PyJWT e cryptography**: librerie crittografiche impiegate per la gestione dell'autenticazione Server-to-Server. Hanno permesso di strutturare i claim dei token JWT e di firmare digitalmente il payload utilizzando chiavi asimmetriche RSA con algoritmo di hashing RS512, requisito stringente per l'interfacciamento con la piattaforma Interacta.

2.3 Scripting Bash e Ambiente Linux (Cron, Logrotate)

L'intera infrastruttura di monitoraggio e i relativi script di automazione sono stati distribuiti e messi in produzione su una macchina virtuale basata su sistema operativo Ubuntu Linux. La scelta dell'ambiente Linux ha permesso di sfruttare nativamente potenti strumenti a riga di comando e demoni di sistema per orchestrare i flussi di dati e ottimizzare i processi eseguiti in background.

Scripting Bash e Gestione dei Permessi

Il linguaggio Bash è stato impiegato per la realizzazione di script leggeri, focalizzati sull'interazione diretta con il sistema operativo o per fare da ponte (wrapper) tra l'interfaccia di Zabbix e gli applicativi complessi scritti in Python. Un caso d'uso primario ha riguardato lo sviluppo dei *Custom AlertScripts*, posizionati in specifiche directory di sistema come `/usr/lib/zabbix/alertscripts/`. Un esempio rilevante è lo script `ScriptCustomExportProblemsToCSV.sh`, progettato per intercettare i parametri degli allarmi che Zabbix passa come variabili d'ambiente (tramite le Macro come `$1`, `$2...`) e accodarli in un file CSV strutturato.

Un aspetto critico affrontato in ambiente server è stata la rigorosa gestione dei permessi: i processi automatizzati eseguiti dall'utente di sistema **zabbix** operano in un perimetro isolato per motivi di sicurezza. Per consentire al demone di scrivere l'output all'interno di una directory protetta dell'utente principale (come `/home/administrator/`), è stato necessario configurare dei link simbolici (Sym-link) o applicare *Access Control Lists* (ACL), aggirando l'errore di *Permission Denied* senza compromettere le politiche di isolamento Linux.

Orchestrazione dei Flussi tramite Crontab e Systemd

Per disaccoppiare i carichi di calcolo analitico e di reportistica dal ciclo di vita vitale del server Zabbix, le esecuzioni sono state delegate direttamente al sistema operativo tramite due approcci complementari:

- **Crontab:** Impiegato per l'orchestrazione dei task in modalità batch (periodica). Ad esempio, l'invio automatizzato del report giornaliero dei KPI è stato schedulato alle ore 06:10 del mattino inserendo la direttiva `10 6 * * *` per richiamare lo script eseguendo esplicitamente l'interprete Python all'interno del suo *virtual environment* (`venv`).
- **Systemd:** Utilizzato per trasformare i middleware di ascolto continuo (come lo script asincrono per gli eventi ISAPI delle telecamere Hikvision) in demoni nativi del sistema. Questo garantisce l'avvio automatico delle dipendenze al boot e l'applicazione di policy di `Restart=always`, essenziali per ripristinare autonomamente il servizio a seguito di crash inattesi della connessione.

Prevenzione della Saturazione e Manutenzione (Logrotate)

L'esecuzione parallela di interfacciamenti API, elaborazioni GIS e demoni in ascolto, genera un volume costante di log testuali a fini di *troubleshooting*. Per scongiurare la saturazione dello spazio su disco e il conseguente stallo del server di produzione, la manutenzione (retention) dei log è stata automatizzata tramite l'utility **Logrotate**.

Sono stati creati file di configurazione dedicati nella directory `/etc/logrotate.d/` (come `hikvision_scripts` e `zabbix-interacta-script`), applicando policy temporali (`weekly`) e dimensionali (es. `maxsize 10M` o `30M`) combinate a direttive di compressione `compress`. Durante la fase di progettazione si è resa necessaria una distinzione tecnica fondamentale:

- Per i log afferenti agli script periodici gestiti da Crontab, è stata configurata la direttiva `create`, la quale provvede a rinominare il vecchio file creandone contestualmente uno nuovo con permessi predefiniti (`0644`).
- Per i log manipolati in tempo reale dal servizio continuo sotto systemd, la rotazione classica comportava la perdita del descrittore (file pointer) da parte di Python. Il problema è stato ovviato implementando il parametro `copytruncate`, che copia il contenuto del log corrente nell'archivio storico e svuota l'originale, consentendo allo script di proseguire la scrittura in modo ininterrotto e invisibile al processo applicativo.

2.4 Interacta e JSON Web Token (JWT)

Figura 2.3: Logo Interacta

Interacta è una piattaforma cloud enterprise progettata per la gestione dei processi aziendali, la collaborazione e il ticketing. All'interno di Secoval S.r.l.,

questo strumento costituisce il fulcro operativo per la gestione delle manutenzioni sul territorio: ogni guasto hardware o anomalia di rete deve tradursi in un ticket assegnato alla squadra tecnica competente.

L'obiettivo dell'integrazione è automatizzare questo flusso, eliminando il passaggio umano tra il rilevamento del guasto e l'apertura della segnalazione. Il sistema di monitoraggio Zabbix è stato configurato per interfacciarsi con le API REST di Interacta, inviando un payload JSON strutturato ogni volta che un *Trigger* critico cambia stato (generazione del problema o sua risoluzione).

Essendo una comunicazione di tipo Server-to-Server tra un'infrastruttura on-premise (la VM Zabbix) e un servizio cloud, il requisito di sicurezza primario è l'autenticazione robusta delle richieste HTTP. Per questo scopo, Interacta adotta lo standard **JSON Web Token (JWT)**, definito dalla direttiva RFC 7519. Un JWT è un formato compatto e sicuro per la trasmissione di informazioni (claim) tra due parti, la cui integrità è garantita da una firma crittografica.

L'architettura di sicurezza implementata nel progetto prevede l'uso di crittografia asimmetrica:

- **Algoritmo RS512:** La firma del token avviene utilizzando l'algoritmo RSA combinato con la funzione di hash SHA-512.
- **Gestione delle Chiavi:** È stata generata una coppia di chiavi asimmetriche a 2048 bit. La chiave pubblica è configurata sul portale Interacta per la verifica, mentre la chiave privata risiede in locale sul server Zabbix, protetta da permessi restrittivi.
- **Costruzione del Payload:** Uno script Python dedicato compila i claim obbligatori richiesti da Interacta (**iss** per l'emittente, **sub** per il soggetto, **iat** per il timestamp di emissione, **exp** per la scadenza tipicamente fissata a pochi minuti) e firma l'oggetto tramite la libreria Python **cryptography**.

Il processo di integrazione è gestito da un *Custom AlertScript* (`interactaCreateOrSolveTicket`) richiamato da Zabbix. Lo script riceve in ingresso i parametri dell'evento, genera dinamicamente il token JWT valido per la singola transazione, ed esegue la chiamata POST. Tutte le transazioni e gli eventuali codici di errore HTTP (es. errori 500 lato server) vengono tracciati nel file di log dedicato `interactaCreateOrSolveTicket_log.log` sottoposto a politiche di rotazione rigorose per evitare la saturazione del disco della macchina virtuale.

Capitolo 3

Architettura e Configuration Management

3.1 I CMDB aziendali (Excel) come Source of Truth

Nell'ingegneria dei sistemi informativi enterprise, il Configuration Management Database (CMDB) rappresenta il pilastro fondamentale per la governance IT, configurandosi come l'archivio centralizzato che contiene tutte le informazioni relative ai Configuration Item (CI), ovvero gli asset hardware e software che compongono l'infrastruttura aziendale. Nel contesto della Smart City gestita da Secoval S.r.l., la configurazione e la mappatura dei dispositivi dislocati sul territorio (telecamere IP, Access Point, gateway LTE, switch di rete) erano storicamente frammentate in flussi documentali manuali.

Per abilitare un'architettura di monitoraggio automatizzata basata su Zabbix, è stato necessario identificare una **Single Source of Truth** (Sorgente Unica della Verità), ovvero un'entità informativa autoritativa a cui il sistema potesse attingere per allineare lo stato logico dei controlli con lo stato fisico degli asset sul territorio.

Struttura e Ruolo dei Due File Excel Aziendali

Sebbene i framework metodologici avanzati (come ITIL) prevedano l'adozione di software CMDB relazionali dedicati, la realtà operativa di molte medie imprese e utility locali vede l'utilizzo consolidato di fogli di calcolo elettronici per la gestione degli inventari. Secoval gestisce la conoscenza della propria infrastruttura Smart City attraverso **due file Excel principali**, che fungono a tutti gli effetti da CMDB operativi:

1. **Excel degli Asset di Videosorveglianza (TVCC):** Questo documento contiene l'inventario analitico di tutte le telecamere IP (prevalentemente Hikvision) installate nei vari comuni della Comunità Montana. Per ogni apparato sono mappati dati cruciali quali l'indirizzo IP statico, l'univoco ID **Host** aziendale, il costruttore, il modello e, dato fondamentale per la manutenzione fisica, l'**ID Palo** o l'identificativo della cassetta stradale di derivazione all'interno della quale è alloggiato il dispositivo.

2. **Excel degli Asset di Connettività e Networking:** Questo secondo file cataloga l'infrastruttura di rete che abilita il trasporto del dato, contenente i record relativi agli Access Point Wi-Fi pubblici, ai router di frontiera, ai gateway LTE utilizzati per il rilegamento wireless dei siti isolati e agli switch industriali da campo. Anche in questo caso, la struttura associa a ogni IP i relativi metadati logici e geografici.

Questi due file sono depositati all'interno del file system della macchina virtuale Ubuntu Server dedicata al monitoraggio, all'interno della directory di lavoro del progetto (`/home/administrator/zabbixHostsScript/`). Essi vengono aggiornati periodicamente dal personale tecnico di Secoval non appena un nuovo apparato viene installato o censito sul campo.

Il Processo di Estrazione della Conoscenza

Il limite insito nell'uso dei fogli di calcolo come CMDB è la loro staticità: Zabbix non è in grado di leggere nativamente un file `.xlsx` per configurare i propri controlli. Per risolvere questo problema e automatizzare il flusso, è stata ingegnerizzata un'architettura di sincronizzazione software in linguaggio Python.

Il sistema opera secondo una logica sequenziale basata su tre fasi macro-operative (ETL - Extraction, Transformation, Loading):

- **Estrazione (Extraction):** Lo script Python, sfruttando l'efficienza di librerie dedicate alla manipolazione dati come `pandas` e `openpyxl`, accede ai percorsi assoluti dei due file Excel e ne legge le righe in modo asincrono. Questa operazione permette di estrarre la conoscenza testuale e strutturata racchiusa nelle celle (indirizzi IP, ID Palo, denominazione del Comune).
- **Trasformazione (Transformation):** I dati grezzi estratti dagli Excel vengono normalizzati e mappati all'interno di dizionari ed entità computazionali Python. In questa fase, ad esempio, lo script associa a ciascun dispositivo il corretto **Host Group** di Zabbix (es. separando logicamente il gruppo `SmartCity/TVCC` dal gruppo `SmartCity/Connectivity`) e determina quale **Template** di monitoraggio applicare in base al modello hardware rilevato nel foglio elettronico.
- **Caricamento (Loading via API JSON-RPC):** La conoscenza strutturata e arricchita viene infine iniettata all'interno del server Zabbix. Lo script Python effettua chiamate HTTP POST verso l'endpoint delle API native di Zabbix, utilizzando il protocollo leggero **JSON-RPC**. Il sistema esegue un confronto dinamico: se un host presente nell'Excel non esiste su Zabbix, viene creato ex-novo valorizzando i campi dell'inventario (inclusi i metadati aziendali come l'ID Palo); se l'host esiste già ma presenta discrepanze (es. è cambiato l'IP o la posizione), viene aggiornato; se l'host è stato rimosso dall'Excel, viene disattivato o rimosso dal monitoraggio.

Questa scelta progettuale ha consentito di salvaguardare l'operatività quotidiana dell'azienda – che continua a utilizzare i familiari fogli Excel come interfaccia di inserimento dati – e al contempo di garantire i requisiti rigorosi di un'architettura ITID, trasformando un archivio statico manuale nella vera forza motrice (*Source of Truth*) dell'automazione del monitoraggio.

3.2 Sviluppo del motore di Provisioning in Python via Zabbix API e setup della Macchina Virtuale Ubuntu

Per tradurre in realtà l'astrazione del CMDB basato su Excel (analizzato nella sezione precedente) e automatizzare il popolamento dell'infrastruttura, è stato necessario progettare un ambiente di esecuzione dedicato. L'architettura prevede l'impiego di una Macchina Virtuale (VM) basata su sistema operativo Ubuntu Linux, ospitata sull'infrastruttura di virtualizzazione aziendale (Nutanix). La scelta di Ubuntu Server garantisce stabilità, leggerezza e una perfetta compatibilità con gli stack di automazione open-source tipici degli ambienti cloud-native.

Setup dell'Ambiente Operativo e Sicurezza

La configurazione della VM è stata guidata dal principio del minimo privilegio (*Principle of Least Privilege*). A livello di sistema operativo, sono stati definiti contesti di esecuzione nettamente separati per proteggere l'integrità del motore:

- L'utente **administrator**: impiegato per le fasi di sviluppo, amministrazione SSH e per l'aggiornamento dei file Excel di input. L'intera *codebase* risiede all'interno della sua directory di lavoro, specificamente nel percorso `/home/administrator/zabbixHostsScript/`.
- L'utente di sistema **zabbix**: utilizzato dal demone del server di monitoraggio per l'esecuzione in background (*headless*) degli script di alert e di integrazione.

Un vincolo ingegneristico fondamentale per far coesistere i due utenti ed evitare fallimenti critici è stato l'utilizzo esclusivo di percorsi assoluti (*Absolute Paths*) all'interno del codice Python. Questa scelta ovvia all'imprevedibilità della *Current Working Directory* (CWD) quando i task vengono scatenati in background dal motore Zabbix.

L'ecosistema Python e il Virtual Environment (venv)

Il motore di automazione (provisioning) è stato interamente sviluppato in linguaggio Python 3. In un ambiente server di produzione, l'installazione di librerie esterne a livello globale (tramite `sudo pip`) è considerata una pratica da evitare (anti-pattern). Questa operazione rischia infatti di sovrascrivere moduli necessari al sistema operativo, generando conflitti noti come *Dependency Hell*.

Per garantire l'isolamento e la futura scalabilità del sistema, è stato istanziato un *Virtual Environment* (**venv**) dedicato. Come tracciato nella documentazione architetturale, al suo interno sono state installate le dipendenze essenziali:

- **pandas** e **openpyxl** per l'ingestione, la pulizia (Data Preprocessing) e la manipolazione dei dataset Excel.
- **pyzabbix** (o, a livello più basso, la libreria **requests**), un wrapper Python fondamentale per astrarre la complessità delle chiamate HTTP e costruire agilmente le richieste verso le API.

Il Motore di Provisioning e l'Integrazione JSON-RPC

Il core del sistema è un modulo Python progettato con una logica di **Idempotenza**: la sua esecuzione, se ripetuta più volte senza che i dati in ingresso (il file Excel) siano mutati, non altera lo stato del sistema e non genera dispositivi duplicati.

La comunicazione bidirezionale con Zabbix avviene tramite le sue API native, che espongono metodi basati sul protocollo leggero **JSON-RPC**. L'autenticazione è demandata a un API Token univoco (Bearer Token) generato sul frontend, il quale garantisce l'autorizzazione alle chiamate HTTP senza necessità di trasmettere e validare le credenziali a ogni richiesta. L'evoluzione architetturale futura, descritta nei documenti di *Troubleshooting*, prevede di disaccoppiare questo token dal codice sorgente, iniettandolo dinamicamente a runtime tramite variabili d'ambiente (file `.env`) o macro cifrate Zabbix Vault per mitigare i rischi legati all'hardcoding.

Il flusso logico del middleware di provisioning (fase di Loading) segue un ciclo ben definito per ogni riga dell'Excel:

1. **Interrogazione (`host.get`):** Lo script invia una query JSON-RPC per ottenere l'elenco degli host già configurati, filtrandoli per indirizzo IP o nome.
2. **Valutazione (`Diffing`):** Viene eseguito un confronto logico in memoria tra lo stato desiderato (CMDB Excel) e lo stato reale (Zabbix).
3. **Esecuzione (`host.create` o `host.update`):**
 - Se si rileva l'aggiunta di un nuovo dispositivo, Python impacchetta il payload JSON per la chiamata `host.create`.
 - Il sistema compila autonomamente l'Host Group di appartenenza (es. separando il dominio *Networking* da quello *TVCC*) e associa il **Template** corretto, assicurando che la telecamera erediti immediatamente gli item asincroni (Zabbix Trapper) necessari per intercettare gli allarmi ISAPI.
 - Infine, il sistema popola automaticamente i campi dell'*Inventory Mode* del dispositivo. L'inserimento dinamico di metadati come le coordinate geografiche (Latitudine e Longitudine) o l'ID del palo fisico, si rivelerà il tassello fondamentale per la successiva costruzione dei *JSON Web Token* e la creazione contestualizzata dei ticket sulla piattaforma Interacta.

```

1 from pyzabbix import ZabbixAPI
2
3 # Inizializzazione della connessione JSON-RPC
4 zapi = ZabbixAPI("http://localhost/zabbix")
5 zapi.login(api_token="<ZABBIX_API_TOKEN>")
6
7 def provision_smartcity_device(hostname, ip, group_id,
8     template_id, id_palo):
9     # Idempotenza: verifica dell'esistenza pregressa dell'host
10    if not zapi.host.get(filter={"host": hostname}):
11        try:
```

```

11         # Compilazione del payload e chiamata di creazione
12         zapi.host.create(
13             host=hostname,
14             interfaces=[{
15                 "type": 1, "main": 1, "useip": 1,
16                 "ip": ip, "dns": "", "port": "10050"
17             }],
18             groups=[{"groupid": group_id}],
19             templates=[{"templateid": template_id}],
20             inventory_mode=0, # Abilita la gestione manuale
dell'inventario via API
21             inventory={"location": id_palo}
22         )
23         print(f"[OK] Host {hostname} inglobato nel sistema.")
24     except Exception as e:
25         print(f"[ERROR] Fallimento nel provisioning di {
hostname}: {e}")
26     else:
27         print(f"[SKIP] Host {hostname} gia' monitorato. Nessuna
azione intrapresa.")

```

Listing 3.1: Snippet concettuale del motore di provisioning Python tramite Zabbix API

L'implementazione di questo ecosistema Python-Ubuntu ha permesso di disaccoppiare la gestione operativa dell'inventario (eseguita dai tecnici comunali sui file Excel) dalla complessa configurazione sistemistica di Zabbix, concretizzando un approccio embrionale di *Infrastructure as Code* (IaC) adattato alla realtà di un'utility territoriale.

3.3 Integrazione GIS: Estrazione coordinate tramite Excel e visualizzazione in Zabbix

Un elemento distintivo dei progetti in ambito Smart City è la forte connotazione geografica degli asset. Nel contesto del territorio gestito da Secoval, conoscere lo stato logico di un apparato (es. *UP* o *DOWN*) è un'informazione incompleta se non è associata alla sua esatta ubicazione fisica. L'integrazione di logiche GIS (*Geographic Information System*) all'interno del sistema di monitoraggio diventa quindi un requisito architetturale fondamentale per ottimizzare l'intervento sul campo delle squadre di manutenzione e abbattere il *Mean Time To Resolution* (MTTR).

Il flusso dei dati spaziali: dal CMDB (Excel) all'API

La sorgente primaria per i dati spaziali è rappresentata dai file Excel aziendali (il CMDB descritto nella Sezione 3.1). Durante le fasi di installazione o censimento, gli operatori sul campo registrano le coordinate GPS (Latitudine e Longitudine) di ogni singolo palo, telecamera o Access Point all'interno di colonne dedicate nel foglio di calcolo.

Il motore di automazione Python è stato esteso per includere una pipeline di estrazione e normalizzazione di questi dati geografici. La fase di *Data Cleansing* è cruciale: poiché l'inserimento umano su Excel è prone a errori di formattazione (es. l'utilizzo della virgola al posto del punto per i decimali, o la presenza di

stringhe di testo accidentali), lo script Pandas intercetta questi campi, esegue il *parsing* e forza la conversione dei valori in formato *floating-point* aderente allo standard internazionale WGS 84.

Una volta normalizzate, le coordinate vengono passate al modulo di comunicazione JSON-RPC. Come definito nelle logiche di *provisioning*, Zabbix richiede che l'*Inventory Mode* dell'host sia impostato su 0 (Manuale/API). In questo modo, lo script Python inietta direttamente i valori nei campi nativi `location_lat` e `location_lon` del database Zabbix.

```

1 # Estratto della logica di update dell'inventario host
2 def update_host_location(host_id, latitude, longitude):
3     try:
4         zapi.host.update(
5             hostid=host_id,
6             inventory={
7                 "location_lat": str(latitude),
8                 "location_lon": str(longitude)
9             }
10        )
11    except Exception as e:
12        print(f"Errore durante l'aggiornamento GIS per host {
13            host_id}: {e}")

```

Listing 3.2: Iniezione delle coordinate geografiche tramite API JSON-RPC

Gestione dinamica e Aggiornamento degli Host

Il sistema è progettato per essere tollerante ai cambiamenti (*Idempotenza*). Quando viene aggiunto un nuovo host nell'Excel, lo script lo rileva e, contestualmente all'operazione di `host.create`, ne inietta le coordinate.

Tuttavia, il reale valore ingegneristico si manifesta nella gestione delle modifiche (es. una telecamera che viene ricollocata su un incrocio differente). Durante l'esecuzione schedulata, lo script effettua un *diffing* (confronto) tra le coordinate correntemente salvate su Zabbix (ottenute tramite `host.get`) e quelle appena lette dall'Excel. Se viene rilevata una discrepanza ($\Delta > 0$), il sistema lancia una chiamata `host.update` sovrascrivendo la vecchia posizione, garantendo che Zabbix sia sempre un "gemello digitale" perfettamente allineato con la realtà fisica.

Visualizzazione su Zabbix Geomap

Per chiudere il cerchio e fornire un reale supporto operativo alla centrale di controllo, si è sfruttato il widget nativo **Geomap** introdotto nelle recenti versioni di Zabbix.

La Geomap non è una semplice mappa statica, ma una *dashboard interattiva* che funge da aggregatore visivo. Il widget è configurato per appoggiarsi a *Tile Providers* open-source (come OpenStreetMap), interrogando in tempo reale il database per estrarre l'elenco degli host (es. filtrando per Host Group "TVCC" o "Access Point") e i rispettivi campi `location_lat` e `location_lon`.

La potenza di questo strumento risiede nel suo accoppiamento diretto con il motore di valutazione dei **Trigger**:

- **Stato Nominale (OK):** Se tutti gli Item associati al dispositivo (es. Ping, flusso video ISAPI) rientrano nelle soglie attese, l'host viene renderizzato sulla mappa con un *marker* verde.
- **Stato di Anomalia (PROBLEM):** Appena il middleware rileva, ad esempio, un *Video Tampering* e fa scattare il trigger, la propagazione dello stato raggiunge immediatamente la dashboard. Il *marker* sulla mappa cambia istantaneamente colore, assumendo la tinta corrispondente alla *Severity* del guasto (es. giallo per *Warning*, rosso lampeggiante per *High/Disaster*).

Questa interfaccia geografica permette all'operatore di Secoval di ottenere un'immediata *Situational Awareness* (consapevolezza situazionale). A colpo d'occhio è possibile individuare cluster di guasti (es. intere vie o piazze oscurate contemporaneamente), facilitando l'analisi delle cause alla radice (*Root Cause Analysis*) legate a problematiche di fornitura elettrica o cadute dei ponti radio dorsali che alimentano quelle specifiche aree territoriali.

3.4 Organizzazione gerarchica in Host Groups degli host e i codici comunali

La gestione su larga scala di un'infrastruttura IoT e di rete distribuita sul territorio, come quella di una Smart City, impone la necessità di adottare paradigmi di classificazione logica rigorosi. Un approccio "piatto" (*flat*), in cui tutti i dispositivi (Configuration Item) risiedono in un unico contenitore logico o vengono raggruppati senza un criterio formale, rende impossibile l'analisi aggregata degli allarmi, complica l'assegnazione dei permessi e vanifica il calcolo di KPI specifici per singola area geografica.

Il Paradigma dei Nested Host Groups in Zabbix

Per risolvere questa criticità architetturale, si è sfruttata la funzionalità nativa di Zabbix relativa ai **Nested Host Groups** (Gruppi di Host annidati). Questa feature permette di creare alberature gerarchiche e *namespace* logici separando i livelli tramite il carattere *slash* (/).

In fase di analisi e progettazione congiunta con i sistemisti di Secoval, è stata definita una tassonomia standardizzata basata su tre livelli gerarchici principali:

1. **Dominio di Progetto:** Il nodo radice (es. **SmartCity** o **Corporate**), che disaccoppia logicamente i servizi esposti al pubblico e sul territorio dai server interni aziendali.
2. **Dominio Tecnologico:** Il ramo funzionale, che separa le tipologie di apparato (es. **SmartCity/TVCC** per l'ecosistema di videosorveglianza Hikvision, **SmartCity/Networking** per switch e Access Point).
3. **Dominio Geografico / Amministrativo:** Il nodo foglia, rappresentato dal **Codice Comunale** o dalla denominazione normalizzata del comune di pertinenza.

Gestione dei Codici Comunali e Normalizzazione (Data Cleansing)

Poiché Secoval opera come partner tecnologico per molteplici amministrazioni comunali (territorio della Valle Sabbia e limitrofi), la separazione per ente è il requisito fondamentale per la reportistica direzionale e per la corretta fatturazione dei servizi legati agli SLA.

Tuttavia, l'inserimento manuale del nome del comune nei file Excel (che fungono da CMDB) espone il sistema a refusi o variazioni di nomenclatura (es. "Roè Volciano" contro "Roe' Volciano" o "Roe Volciano"). Per garantire l'integrità referenziale all'interno del database relazionale di Zabbix, lo script Python di *provisioning* implementa una logica di normalizzazione (*Data Cleansing*): mappa le stringhe testuali eterogenee lette dall'Excel in **Codici Comunali** univoci e standardizzati (ad esempio derivandoli dal codice catastale Belfiore o da codifiche interne prestabilite).

Automazione dell'Alberatura via API

Durante il ciclo di caricamento (*Loading*) verso il server, il middleware Python non si limita ad assegnare l'host a un gruppo preesistente, ma gestisce dinamicamente l'intera alberatura del sistema.

Per ogni riga elaborata dal dataframe Pandas, lo script calcola la stringa del gruppo di destinazione, assemblandola dinamicamente (es. `SmartCity/TVCC/Vestone`). Successivamente, interroga il server tramite API JSON-RPC richiamando il metodo `hostgroup.get`. Se la risposta indica che il gruppo non esiste (uno scenario tipico quando si acquisisce la gestione di un nuovo comune), lo script invoca asincronamente `hostgroup.create` per generare il nuovo ramo gerarchico. Solo a seguito di questa validazione topologica, il dispositivo viene inglobato e associato all'`hostgroupid` corretto.

Vantaggi Operativi e Architetture

L'ingegnerizzazione di questa struttura logica, abilitata dall'automazione Python, garantisce all'infrastruttura di Secoval tre vantaggi misurabili:

- **Role-Based Access Control (RBAC) multi-tenant:** Sfruttando i permessi granulari del frontend web di Zabbix, è ora possibile generare account utente limitati (ad esempio per le forze di Polizia Locale di un determinato municipio) che godono di visibilità in sola lettura esclusivamente sul ramo gerarchico del proprio territorio (`SmartCity/TVCC/CodiceComune`), precludendo l'accesso agli asset di altri enti.
- **Filtraggio Dinamico delle Dashboard:** L'interfaccia grafica e i widget operativi (come la *Geomap* implementata per le coordinate GIS o i widget *Problems by severity*) possono essere filtrati in modo macroscopico puntando al nodo radice, oppure focalizzati sul singolo nodo amministrativo.
- **Aggregazione dei KPI:** La gerarchia rigorosa facilita enormemente il lavoro del motore Python deputato all'analisi dei log (descritto nel Capitolo 6). Sfruttando le funzioni di aggregazione (`groupby`), il sistema aggrega

facilmente metriche complesse come l'MTTR o la frequenza di *Flapping* per singolo codice comunale, fornendo *insight* precisi alla direzione tecnica.

Capitolo 4

Implementazione: Controlli Attivi ed Eventi TVCC

4.1 Logiche di Polling vs Trapping in Zabbix

Per comprendere le scelte architetturali adottate durante il tirocinio per il monitoraggio della rete Smart City di Secoval, è fondamentale analizzare come il server Zabbix opera "sotto il cofano" (under the hood) per la raccolta e l'elaborazione delle metriche. Zabbix si basa sul concetto di *Item*, ovvero la singola unità di dato raccolta da un host. L'acquisizione di questi Item può avvenire seguendo due paradigmi diametralmente opposti: il **Polling** (controllo attivo) e il **Trapping** (controllo asincrono o passivo).

Il Paradigma del Polling (Controllo Attivo e Network Discovery)

Nel modello di Polling, è il server Zabbix (tramite i suoi processi interni denominati *poller*) a prendere l'iniziativa. A intervalli di tempo regolari e predefiniti (*Update interval*), il server apre una connessione verso l'host target, interroga il dispositivo (tramite protocolli come ICMP, SNMP, o richieste HTTP) e attende una risposta.

Nella fase iniziale del progetto in Secoval, questa logica è stata sfruttata pesantemente per l'individuazione automatica degli apparati sul territorio tramite le **Discovery Rule**. Zabbix era configurato per lanciare un "ICMP Ping" orario su intere sottoreti (subnet) al fine di scovare nuovi nodi (TVCC, Access Point, Switch, Gateway LTE).

Il flusso di automazione nativo procedeva in questo modo:

1. **Discovery Rule:** Il server scansiona la sottorete. Se un indirizzo IP risponde al ping, viene generato un evento di "Discovery".
2. **Discovery Action:** Un set di regole logiche intercetta l'evento. Se l'IP risponde e magari presenta specifiche porte aperte, l'Action provvede ad aggiungere automaticamente l'host al sistema.
3. **Host Groups e Templates:** L'host appena scoperto viene inserito in un gruppo logico (es. **SmartCity/TVCC**) e gli viene associato un **Template**. Il template è un "modello" preconfezionato che contiene tutti gli *Item*, i

Trigger e i grafici specifici per quella tecnologia, garantendo che ogni telecamera erediti le stesse logiche di monitoraggio senza configurazioni manuali ripetitive.

I limiti del Polling riscontrati nel progetto: Nonostante l'efficacia del Network Discovery nativo, abbiamo riscontrato due limiti architetturali critici. In primo luogo, il discovery basato su ICMP creava host nominati univocamente con il loro indirizzo IP (o DNS name generico), privandoli del contesto aziendale vitale (come l'ID Host o l'ID Palo). In secondo luogo, il polling introduce una latenza intrinseca: se l'update interval di un controllo è di 5 minuti, un guasto che si verifica al minuto 1 verrà rilevato solo al minuto 5. Per eventi di sicurezza critici (come l'oscuramento di una telecamera), questo ritardo non è accettabile.

Il Paradigma del Trapping (Controllo Asincrono e Zabbix Sender)

Per superare i limiti del polling sugli eventi critici, il progetto ha implementato il paradigma del **Trapping**. Nel trapping, il server Zabbix non interroga il dispositivo, ma rimane in ascolto passivo (tipicamente sulla porta TCP 10051 tramite i processi *trapper*). È l'host periferico (o un middleware intermedio) a prendere l'iniziativa e a inviare (*push*) proattivamente il dato al server nel momento esatto in cui l'evento si verifica.

In Zabbix, questo si traduce nell'utilizzo di Item di tipo **Zabbix Trapper**. Tali item non hanno un intervallo di aggiornamento; si limitano ad attendere che l'utility a riga di comando `zabbix_sender` (o una libreria API equivalente) inietti un valore formattato con la tripla: `<Hostname> <Item Key> <Value>`.

Questa logica è stata la chiave di volta per integrare le telecamere **Hikvision**. Poiché le telecamere non possedevano un agent Zabbix nativo in grado di inviare trap, è stato sviluppato uno script Python (Middleware) che:

- Mantiene una connessione HTTP in streaming (ISAPI) con la telecamera.
- Intercetta istantaneamente gli eventi XML/JSON generati dall'hardware (es. *Video Tampering* o *Motion Detection*).
- Traduce l'evento e lancia in background un comando `zabbix_sender` indirizzato al server Zabbix, valorizzando l'item Trapper associato a quello specifico dispositivo.

Il risultato è un monitoraggio in tempo reale a latenza zero, impossibile da ottenere con il solo Polling, riducendo peraltro il traffico di rete e il carico di calcolo della CPU sul server Zabbix (che non deve più ciclare richieste a vuoto su centinaia di host).

Trigger e Motore di Valutazione (History Cache e Alarms)

Una volta che il dato raggiunge Zabbix, indipendentemente dal fatto che derivi da un Polling o da un Trapping, viene immagazzinato nella RAM (History Cache) per un'elaborazione ultra-rapida. È qui che entrano in gioco i **Trigger**.

Sotto il cofano, un Trigger è un'espressione logica e matematica che valuta lo storico recente dei dati di un Item per determinare se l'host si trova in uno stato

di "OK" o "PROBLEM". Ad esempio, un trigger potrebbe avere una logica simile a:

$$\text{last}(/Host/icmp.ping) = 0$$

Questa espressione stabilisce che se l'ultimo valore raccolto dal ping è 0 (host non raggiungibile), il trigger cambia il suo stato.

I trigger in Zabbix sono dotati di funzionalità avanzate cruciali per il nostro ambiente Smart City:

- **Isteresi (Hysteresis):** Permette di definire soglie differenti per l'attivazione e la risoluzione del problema, fondamentale per evitare il fenomeno del *Flapping* su reti LTE instabili (dove il ping potrebbe fallire e tornare ciclicamente in pochi secondi).
- **Gestione degli Eventi e Macros:** Quando un trigger scatta, Zabbix genera un *Event* univoco (es. {EVENT.ID}). Il motore interno associa automaticamente al problema tutta una serie di metadati tramite le **Macro** (variabili di sistema come {HOST.NAME}, {TRIGGER.NAME}, {EVENT.DATE}).

L'ultimo anello della catena architetturale è l'**Action**. Quando un trigger rileva un'anomalia, l'Action intercetta il cambiamento di stato. Nel contesto dell'integrazione Secoval, l'Action non si limita a inviare una banale e-mail, ma esegue un *Custom AlertScript* in Bash/Python. A tale script vengono passate le Macro generate dal Trigger come argomenti in input (\$1, \$2...), fornendo al programma tutto il contesto necessario per compilare il JSON Web Token e invocare le API del cloud di ticketing **Interacta** per gestire l'intervento dei tecnici sul campo.

4.2 Sviluppo del Middleware e Ascolto degli Stream Hikvision ISAPI

Il monitoraggio tradizionale basato sul protocollo ICMP (Ping) è in grado di certificare la presenza in rete di un dispositivo, ma risulta cieco di fronte a guasti di natura applicativa o fisica. Nel contesto della videosorveglianza urbana (TVCC), una telecamera potrebbe rispondere regolarmente al ping ma risultare oscurata, manomessa o ruotata. Per ottenere una visibilità reale sullo stato di salute degli apparati, è stato necessario spostare il monitoraggio dal livello di rete (L3) al livello applicativo (L7).

Le telecamere Hikvision in dotazione a Secoval S.r.l. espongono le proprie funzionalità tramite **ISAPI** (*Intelligent Security API*), un'interfaccia RESTful basata su HTTP. L'obiettivo ingegneristico è stato quello di creare un *middleware* in grado di configurare massivamente questi apparati e di restare in ascolto dei loro allarmi in tempo reale, traducendoli in metriche compatibili con Zabbix.

Setup e Configurazione tramite Payload XML

Affinché una telecamera generi un evento, le funzioni di analisi video (*Smart Events*) devono essere prima abilitate e configurate. La sfida principale consisteva

nel superare la configurazione manuale tramite interfaccia web, impraticabile per centinaia di dispositivi.

È stato quindi sviluppato un meccanismo di *provisioning* che, tramite richieste HTTP di tipo PUT, invia specifici payload in formato XML agli endpoint ISAPI della telecamera. Ad esempio, per abilitare l'allarme di manomissione (*Video Tampering*), il sistema trasmette un documento strutturato al percorso `/ISAPI/Smart/VideoTampering/1:`

```

1 <VideoTampering version="2.0" xmlns="http://www.isapi.org/ver20/
  XMLSchema">
2   <enabled>true</enabled>
3   <videoTamperingLevel>2</videoTamperingLevel>
4   <RegionList>
5     <Region>
6       <id>1</id>
7       <RegionCoordinatesList>
8         <RegionCoordinates>
9           <positionX>0</positionX>
10          <positionY>0</positionY>
11        </RegionCoordinates>
12        <RegionCoordinates>
13          <positionX>1000</positionX>
14          <positionY>1000</positionY>
15        </RegionCoordinates>
16      </RegionCoordinatesList>
17    </Region>
18  </RegionList>
19 </VideoTampering>

```

Listing 4.1: Esempio di payload XML per l'attivazione del Video Tampering

Nella prima iterazione del progetto, l'invio di queste configurazioni è stato demandato a script Bash ausiliari (come `configura_telecamere.sh`) orchestrati da Python tramite la libreria `subprocess`. Tuttavia, come evidenziato nell'analisi prestazionale, questo approccio introduce un overhead di sistema. Un'evoluzione architetturale futura prevede l'implementazione di chiamate native asincrone tramite `aiohttp` per eliminare i colli di bottiglia legati all'apertura continua di nuovi processi shell.

Il Middleware Python e l'ascolto dell'Alert Stream

Una volta configurate, le telecamere sono pronte a generare allarmi. Invece di interrogare continuamente la telecamera per chiedere se ci siano novità (*Polling*), abbiamo optato per un approccio *Event-Driven* (basato su *Trapping*). Questo azzerà la latenza di rilevamento e minimizza il traffico di rete.

Il cuore di questa integrazione è uno script Python, eseguito come demone (*systemd service*) in background sul server. Lo script instaura una connessione HTTP persistente in streaming verso l'endpoint `/ISAPI/Event/notification/alertStream`. Finché la connessione è aperta, la telecamera "spinge" proattivamente (push) pacchetti XML (o multipart) ogni volta che un sensore scatta.

Di seguito si riporta un estratto semplificato della logica core del middleware in ascolto:

```

1 import requests
2 import subprocess
3 import xml.etree.ElementTree as ET

```

```

4 from requests.auth import HTTPDigestAuth
5
6 def listen_to_camera_stream(camera_ip, user, password,
7     host_zabbix):
8     url = f"http://{camera_ip}/ISAPI/Event/notification/
9     alertStream"
10
11     try:
12         # Apertura della connessione persistente (stream=True)
13         response = requests.get(url, auth=HTTPDigestAuth(user,
14             password), stream=True, timeout=60)
15
16         # Iterazione continua sui chunk di dati in arrivo
17         for line in response.iter_lines():
18             if line:
19                 decoded_line = line.decode('utf-8')
20
21                 # Intercettazione dell'inizio di un blocco XML
22                 if "<EventNotificationAlert" in decoded_line:
23                     # Logica di buffering (omessa per brevit )
24                     # Parsing dell'XML ricevuto
25                     root = ET.fromstring(xml_buffer)
26                     event_type = root.find('..eventType').text
27                     event_state = root.find('..eventState').text
28
29                     # Se l'evento   un Tampering o un Motion
30                     Detection
31                     if event_type in ["videoloss", "tampering", "
32                     VMD"]):
33                         # Traduzione per Zabbix: active=1,
34                         inactive=0
35                         value = "1" if event_state == "active"
36                     else "0"
37
38                     # Inoltro asincrono a Zabbix tramite
39                     Zabbix Sender
40                     send_to_zabbix(host_zabbix, event_type,
41                         value)
42
43         except requests.exceptions.RequestException as e:
44             print(f"Errore di connessione allo stream di {camera_ip}:
45             {e}")
46
47 def send_to_zabbix(zabbix_host, item_key, value):
48     # Costruzione del comando zabbix_sender
49     cmd = [
50         "zabbix_sender",
51         "-z", "127.0.0.1",
52         "-s", zabbix_host,
53         "-k", item_key,
54         "-o", value
55     ]
56     # Esecuzione isolata per non bloccare lo stream
57     subprocess.Popen(cmd, stdout=subprocess.DEVNULL, stderr=
58         subprocess.DEVNULL)

```

Listing 4.2: Middleware Python per l'ascolto persistente dello stream ISAPI

Motivazioni Architetture e Sicurezza

L'implementazione di questo listener continuo in Python ha rappresentato una svolta prestazionale. La scelta di utilizzare `subprocess.Popen` al posto di `subprocess.run` per l'invio del dato tramite `zabbix_sender` è fondamentale: consente al processo Python di "sganciare" l'invio della notifica (modalità **fire-and-forget**), tornando immediatamente in ascolto sullo stream senza perdere eventuali eventi ravvicinati.

Inoltre, operando nel contesto di un ambiente di produzione Linux rigoroso, lo script è configurato per utilizzare percorsi assoluti ed è eseguito all'interno del proprio *Virtual Environment* (`venv`). Come documentato internamente, l'eseguibile lavora senza elevare i privilegi (`su zabbix zabbix`), delegando la gestione degli errori e la rotazione dei log all'utility di sistema per prevenire saturazioni del disco causate dall'elevato *throughput* dei messaggi XML elaborati.

4.3 Inoltro asincrono tramite Zabbix Sender

Una volta che il middleware Python ha intercettato e decodificato un evento critico (come il *Video Tampering*) dallo stream ISAPI della telecamera, emerge la necessità di notificare l'anomalia al server Zabbix nel minor tempo possibile. Per soddisfare questo requisito e garantire un monitoraggio in tempo reale, l'architettura si affida a un inoltro dati di tipo *push*.

La tecnologia impiegata per questa operazione è **Zabbix Sender**, un'utility nativa a riga di comando (CLI) progettata per inviare pacchetti di dati direttamente al processo *Zabbix Trapper*, tipicamente in ascolto sulla porta TCP 10051 del server.

A livello implementativo, la chiamata all'eseguibile `zabbix_sender` è gestita dallo script Python, ma in maniera strettamente **asincrona**. Invece di utilizzare funzioni bloccanti, l'invio è demandato al costruttore `subprocess.Popen`, configurato in modalità *fire-and-forget* reindirizzando i flussi di output e di errore standard (`stdout` e `stderr`) verso `subprocess.DEVNULL`.

Le ragioni e le motivazioni ingegneristiche alla base di questa specifica scelta implementativa sono molteplici e documentate nelle linee guida di sistema:

- **Prevenzione del blocco del listener (Non-blocking I/O):** Se lo script Python avesse utilizzato una chiamata sincrona (come `subprocess.run`), il processo si sarebbe sospeso in attesa di un ACK (Acknowledgement) da parte del server Zabbix. In caso di temporanea latenza di rete o sovraccarico del server, questo blocco avrebbe interrotto la lettura dello stream HTTP della telecamera, causando un buffer overflow o la potenziale perdita di allarmi ravvicinati.
- **Latenza Zero:** Demandando l'invio asincrono a un processo figlio separato, l'evento viene "sparato" a Zabbix nella frazione di secondo successiva al parsing dell'XML. Questo garantisce che i tempi di reazione del sistema di monitoraggio siano praticamente nulli.
- **Disaccoppiamento dei componenti:** Il middleware Python mantiene un focus esclusivo e performante sull'ingestione e la traduzione del flusso ISAPI. L'onere di instaurare i socket TCP, negoziare la trasmissione e chiudere la connessione viene delegato al sistema operativo e al binario `zabbix_sender`.

(scritto nativamente in C e altamente ottimizzato), alleggerendo così il carico computazionale della virtual machine.

4.4 Definizione di Trigger e Discovery Actions OPPURE MEGLIO Configurazione Pratica di Trigger e Actions per gli Eventi TVCC

Avendo già analizzato le logiche di base del motore di valutazione di Zabbix (Sezione 4.1), in questa sezione ci si concentrerà sull'implementazione pratica degli Item, dei Trigger e delle Action configurate per orchestrare gli allarmi provenienti dalle telecamere e integrarli con la piattaforma Interacta.

Il superamento delle Discovery Rule tradizionali

Nella fase di prototipazione del sistema, il popolamento degli host (le telecamere e gli AP) era stato affidato alle **Discovery Rule** native di Zabbix. Il sistema effettuava un ping (ICMP) su intere sottoreti; le **Discovery Action** intercettavano l'IP che rispondeva e applicavano automaticamente un Template predefinito.

Tuttavia, come documentato nelle specifiche di progetto, questo approccio è stato scartato. La motivazione architetturale risiede nel fatto che le Discovery Rule native nominavano gli host esclusivamente tramite il loro indirizzo IP o il nome DNS generico, perdendo tutto il contesto aziendale necessario (come l'ID Host e l'ID Palo). Per questo motivo, l'automazione tramite Discovery nativa è stata disabilitata e sostituita integralmente dallo script Python che agisce via API (JSON-RPC) leggendo i file Excel del CMDB aziendale, garantendo così un allineamento perfetto tra il sistema di monitoraggio e l'inventario fisico.

Item Trapper e Trigger per le Telecamere

Le telecamere importate dallo script API vengono associate a un Template specifico creato ad-hoc per i dispositivi Hikvision. All'interno di questo Template, per gestire gli eventi del middleware ISAPI, sono stati definiti degli **Item di tipo Zabbix Trapper** (es. con chiave `hikvision.event.tampering`).

Il **Trigger** associato a questo Item è il vero motore decisionale. Non si limita a valutare la presenza di un guasto, ma deve gestire la transizione di stato per l'apertura del ticket. L'espressione logica applicata è simile alla seguente:

```
last(/Template Hikvision/hikvision.event.tampering)=1
```

Inoltre, per prevenire falsi allarmi dovuti a micro-disconnessioni del flusso video, è stata implementata la funzione `nodata()` all'interno del Trigger. Se il middleware Python smette di inviare dati per un tempo predefinito (es. 5 minuti), il Trigger scatta per segnalare non una manomissione, ma l'irreperibilità del servizio di ascolto stesso.

Trigger Actions e Integrazione con Interacta

Quando un Trigger cambia stato (da OK a PROBLEM o viceversa), entra in gioco il sistema di **Actions**. Invece di configurare un'Action per inviare una semplice no-

tifica e-mail, il sistema è stato istruito per eseguire un'operazione di tipo "Remote Command", puntando al *Custom AlertScript* denominato `interactaCreateOrSolveTicket`.

La motivazione per utilizzare questo paradigma è la possibilità di iniettare dinamicamente il contesto del guasto all'interno dello script Bash/Python. L'Action è configurata per passare allo script i seguenti argomenti tramite le Macro di Zabbix:

- `{EVENT.ID}`: L'identificativo univoco del guasto, essenziale per correlare la futura risoluzione allo stesso ticket aperto in Interacta.
- `{HOST.NAME}` e `{INVENTORY.LOCATION}`: Per comunicare alla piattaforma l'ID dell'host e il palo fisico su cui i tecnici dovranno intervenire.
- `{TRIGGER.STATUS}`: Il valore che indica allo script se inviare alla piattaforma un payload per la "Creazione" (PROBLEM) o per la "Risoluzione" (OK) del ticket.

Questa catena di configurazioni (Item Trapper → Trigger con isteresi → Action con passaggio di Macro) rappresenta il core logico che ha permesso di azzerare l'intervento umano nella gestione degli incidenti dell'infrastruttura Smart City.

Capitolo 5

Integrazione con il Sistema di Ticketing Interacta

5.1 Sicurezza e Autenticazione Server-to-Server

L'implementazione di un sistema di monitoraggio avanzato rappresenta solo il primo passo verso l'automazione dei processi IT aziendali. In Secoval, rilevare tempestivamente un guasto tramite Zabbix (ad esempio, l'oscuramento di una telecamera o la caduta di un Access Point) è inutile se l'informazione non raggiunge in modo strutturato la squadra tecnica incaricata della manutenzione. La piattaforma cloud aziendale utilizzata per coordinare questi interventi sul campo è **Interacta**.

L'integrazione tra Zabbix e Interacta nasce quindi dall'esigenza di colmare il divario tra la fase di *rilevamento* (Fault Detection) e la fase di *risoluzione* (Incident Management).

Obiettivi e Sfide dell'Integrazione

La costruzione di un ponte comunicativo tra l'infrastruttura di monitoraggio (on-premise) e il sistema di ticketing (in cloud) ha richiesto la definizione di obiettivi rigorosi:

- **Apertura "Zero-Touch":** Al verificarsi di un trigger critico, il sistema deve generare un ticket su Interacta senza alcun intervento umano, abbattendo drasticamente i tempi di latenza e, di conseguenza, il *Mean Time To Resolution* (MTTR).
- **Iniezione di Contesto:** Il ticket non deve essere generico, ma deve contenere metadati vitali per il tecnico sul campo (es. l'indirizzo IP, l'ID dell'Host, l'ID del Palo fisico e la descrizione esatta del problema rilevato).
- **Allineamento di Stato (Auto-Recovery):** Se il problema rientra autonomamente (es. sbalzo di tensione temporaneo), Zabbix deve comunicare a Interacta di risolvere il ticket, evitando alle squadre tecniche uscite a vuoto.

Le sfide principali per raggiungere questi obiettivi non sono state unicamente di natura logica, ma soprattutto di natura architetturale e legata alla sicurezza

informatica. Il traffico dati, infatti, deve attraversare la rete pubblica per raggiungere il cloud di Interacta, esponendo il sistema a potenziali attacchi (come il *Man-in-the-Middle* o il *Replay Attack*).

Il Cuore dell'Integrazione: Autenticazione Server-to-Server

In un'applicazione web tradizionale, l'autenticazione si basa su interazioni umane (es. inserimento di username e password) e sul mantenimento di una sessione tramite cookie. Nel nostro scenario, un server Linux (Zabbix) deve dialogare autonomamente con un altro server (Interacta). Questo paradigma, noto come comunicazione **Machine-to-Machine (M2M)** o **Server-to-Server**, richiede standard di sicurezza profondamente diversi.

Inizialmente, l'approccio più banale avrebbe potuto prevedere l'uso di una semplice API Key statica o di una Basic Authentication. Tuttavia, le linee guida di sicurezza di Interacta consigliano caldamente (e spesso impongono per gli ambienti enterprise) l'adozione di un'autenticazione basata su token crittografici dinamici, specificamente lo standard **JSON Web Token (JWT)**.

Le ragioni che hanno portato alla scelta di questo framework crittografico sono fondamentalmente tre:

1. **Assenza di credenziali fisse in transito:** Un'API Key statica, se intercettata, garantisce l'accesso illimitato alle API fino alla sua revoca manuale. L'autenticazione Server-to-Server implementata nel nostro sistema genera un token JWT "usa e getta" per ogni singola chiamata HTTP. Il token ha una durata limitata (tipicamente pochi minuti, impostata tramite il claim `exp`), rendendolo inutile per un attaccante se rubato dopo la sua scadenza.
2. **Crittografia Asimmetrica e Non-Ripudio:** L'integrazione è stata blindata utilizzando l'algoritmo **RS512** (RSA combinato con SHA-512). Questo approccio si basa su una coppia di chiavi:
 - *Chiave Privata:* È custodita esclusivamente e segretamente a bordo del server Zabbix, limitata da permessi Linux rigorosi. Viene utilizzata dallo script Python per *firmare* il payload del token.
 - *Chiave Pubblica:* È stata caricata nel portale amministrativo di Interacta in fase di setup.

Quando Interacta riceve la richiesta di apertura di un ticket, utilizza la chiave pubblica per decifrare la firma. Se l'operazione ha successo, il server cloud ha la certezza matematica del **non-ripudio**: la richiesta non può che essere partita dal nostro server Zabbix, in quanto unico detentore della chiave privata.

3. **Integrità del Dato:** La firma digitale copre l'intero corpo del token (header e payload). Qualsiasi tentativo di alterazione del messaggio durante il transito (es. cambiare l'ID dell'host per aprire ticket falsi) invaliderebbe istantaneamente la firma, portando Interacta a scartare la richiesta con un errore **401 Unauthorized**.

L'adozione di questo modello Server-to-Server, oltre a soddisfare i requisiti di compliance e sicurezza aziendale, ha permesso di progettare uno script di integrazione robusto, in cui Zabbix non deve mai preoccuparsi di gestire sessioni o

cookie, ma si limita a "firmare" le transazioni solo nel momento esatto in cui un disservizio richiede l'attenzione umana.

5.2 Generazione e firma del token JWT (Algoritmo RS512)

Il cuore dell'autenticazione verso la piattaforma Interacta si basa sulla costruzione e sulla firma crittografica di un JSON Web Token (JWT). Questo standard (RFC 7519) permette di trasmettere informazioni sotto forma di oggetto JSON in modo sicuro e inequivocabile, eliminando la necessità di memorizzare credenziali fisse a bordo del sistema di monitoraggio.

Struttura del Token e Scelta dell'Algoritmo

Un JWT standard è composto da tre parti codificate in Base64URL e separate da un punto (.): l'Header, il Payload e la Signature. L'Header definisce la tipologia di token e l'algoritmo impiegato. Il Payload contiene i *Claim*, ovvero le asserzioni e le dichiarazioni sull'entità autorizzata (es. l'identificativo del client, l'istante di emissione e il momento di scadenza). Infine, la Signature costituisce la firma digitale, che garantisce l'integrità del dato in transito.

Le specifiche di integrazione architetturale dettate da Interacta imponevano un elevato standard di sicurezza, richiedendo l'utilizzo dell'algoritmo **RS512** per la fase di firma. L'RS512 è un algoritmo asimmetrico che combina la robustezza della crittografia RSA (con chiavi da 2048 bit) e la resistenza alle collisioni della funzione di hash SHA-512. A differenza di algoritmi simmetrici come l'HS256 (che utilizzano una singola chiave segreta condivisa tra i due server), l'RS512 prevede l'uso di una coppia di chiavi:

- Una **Chiave Privata**, mantenuta segreta a bordo della macchina virtuale Zabbix, utilizzata dal middleware per firmare il token in uscita.
- Una **Chiave Pubblica**, caricata in precedenza sul pannello amministrativo cloud di Interacta, utilizzata dal provider per decifrare l'hash e verificare l'autenticità del token.

Implementazione Architetturale in Python

L'implementazione del sistema di firma è stata realizzata in linguaggio Python, sfruttando la libreria PyJWT in sinergia con il modulo `cryptography` per la gestione sicura delle chiavi RSA. Prima di effettuare la richiesta HTTP (POST) per l'apertura o la risoluzione di un ticket, l'AlertScript invoca una funzione dedicata che genera dinamicamente la stringa crittografata.

Per minimizzare la finestra temporale di vulnerabilità a eventuali attacchi di tipo *Replay Attack* (in cui un attaccante intercetta il token e tenta di riutilizzarlo), il payload è stato configurato in modo rigoroso, definendo un tempo di scadenza (`exp`) estremamente breve, solitamente impostato a 5 minuti dal momento dell'emissione (`iat`).

```
1 import jwt
2 import time
```

```
3 from cryptography.hazmat.primitives import serialization
4 from cryptography.hazmat.backends import default_backend
5
6 def generate_interacta_jwt(private_key_path, issuer_id,
7     subject_id):
8     # 1. Caricamento in memoria della chiave privata dal file PEM
9     with open(private_key_path, "rb") as key_file:
10         private_key = serialization.load_pem_private_key(
11             key_file.read(),
12             password=None,
13             backend=default_backend()
14         )
15
16     current_time = int(time.time())
17
18     # 2. Definizione del payload strutturato con i Claim
19     # obbligatori
20     payload = {
21         "iss": issuer_id,          # Issuer: identificativo di chi
22         "sub": subject_id,        # Subject: a chi si riferisce
23         "iat": current_time,      # Issued At: timestamp di
24         "exp": current_time + 300 # Expiration Time: scadenza
25     }
26
27     # 3. Generazione e firma del token crittografico
28     encoded_jwt = jwt.encode(payload, private_key, algorithm="
29     RS512")
30     return encoded_jwt
```

Listing 5.1: Snippet Python per la generazione e firma del JWT con algoritmo RS512

Ostacoli Architetture e Testing (Troubleshooting)

La messa in opera di questo sistema crittografico ha richiesto il superamento di alcune sfide e "scogli" ingegneristici non banali.

Il primo aspetto critico ha riguardato la gestione e la sicurezza della chiave privata a livello di sistema operativo Linux. Affinché il *Custom AlertScript* di Zabbix potesse accedere in lettura al file PEM senza esporlo a vulnerabilità, è stato necessario applicare politiche di gestione dei permessi stringenti, assegnando la proprietà del file esclusivamente all'utente di sistema `zabbix` (tramite `chown`) e impostando i permessi di lettura restrittivi (`chmod 400`). Configurazioni meno rigorose avrebbero comportato rischi critici per l'azienda o, viceversa, avrebbero bloccato l'esecuzione dello script restituendo errori del tipo *Permission Denied*.

Un secondo ostacolo particolarmente ingannevole si è verificato durante la prima fase di interfacciamento con le API REST di Interacta. Nonostante la corretta formattazione del payload e la validazione crittografica della firma in ambiente di test locale, i tentativi di comunicazione con l'endpoint restituivano costantemente degli errori bloccanti lato server. In un primo momento, questo comportamento è stato imputato a una potenziale formattazione errata della firma o a una libreria non conforme agli standard Base64URL. Tuttavia, un'attenta analisi dei

log di risposta e un confronto diretto con il provider hanno rivelato che l'errore risiedeva in un disservizio temporaneo e in un'anomalia di validazione sul loro backend, e non in un difetto logico dello script Python. Una volta che il servizio in cloud è stato regolarmente sistemato dai loro tecnici, la nostra architettura di autenticazione ha processato le chiamate con successo senza richiedere ulteriori modifiche al codice, confermando la robustezza dell'implementazione sviluppata.

5.3 Sviluppo del Custom AlertScript in Zabbix

Il tassello architetturale che concretizza l'integrazione tra il sistema di monitoraggio e la piattaforma cloud è rappresentato dal motore esecutivo dell'azione: il *Custom AlertScript*. In Zabbix, quando un'Action viene innescata da un Trigger, è possibile demandare la risposta all'esecuzione di uno script personalizzato, a condizione che questo risieda nella directory di sistema protetta, definita dal parametro `AlertScriptsPath` (di default `/usr/lib/zabbix/alertscripts/`).

Per questo progetto è stato sviluppato lo script Python `interactaCreateOrSolveTicket.py`. Il suo compito è fare da "traduttore" e "postino": riceve i dati dell'allarme generato da Zabbix, costruisce il payload JSON, genera il token JWT per l'autenticazione (come descritto nella sezione precedente) e invia la richiesta HTTP POST all'API REST di Interacta.

Gestione dei Parametri in Ingresso e Logica Condizionale

L'interazione tra il demone Zabbix e lo script Python avviene tramite il passaggio di argomenti a riga di comando (`sys.argv`). L'Action di Zabbix è configurata per iniettare dinamicamente il valore delle Macro all'interno del comando di avvio.

Lo script elabora i parametri ricevuti (tra cui `{EVENT.ID}`, `{HOST.NAME}`, e `{TRIGGER.STATUS}`) e implementa una logica condizionale fondamentale per il ciclo di vita del ticket:

- **Fase di Creazione:** Se il `{TRIGGER.STATUS}` è pari a `PROBLEM`, lo script compila un payload destinato all'endpoint di "Creazione Post", inserendo nel corpo del messaggio i dettagli dell'host, l'ID Palo e la natura del guasto. L'`{EVENT.ID}` generato da Zabbix viene salvato all'interno del ticket come chiave univoca di correlazione.
- **Fase di Risoluzione:** Se il `{TRIGGER.STATUS}` passa a `OK` (o `RESOLVED`), lo script interroga le API di Interacta cercando il ticket che contiene quello specifico `{EVENT.ID}`. Una volta individuato, invia un nuovo payload per forzare la transizione di stato del ticket verso "Chiuso/Risolto", azzerando il carico di lavoro manuale per la centrale operativa.

```
1 import sys
2 import logging
3 import requests
4
5 # Configurazione del file di log con percorsi assoluti
6 LOG_FILE = "/usr/lib/zabbix/alertscripts/
    interactaCreateOrSolveTicket_log.log"
7 logging.basicConfig(filename=LOG_FILE, level=logging.INFO,
```

```

8         format='%(%asctime)s - %(levelname)s - %(
message)s')
9
10 def main():
11     try:
12         # Lettura degli argomenti passati dall'Action di Zabbix
13         trigger_status = sys.argv[1] # es. "PROBLEM" o "OK"
14         event_id = sys.argv[2]        # es. "123456"
15         host_name = sys.argv[3]       # es. "TVCC-ComuneA-01"
16
17         logging.info(f"Ricevuto evento {event_id} per host {
host_name} con stato {trigger_status}")
18
19         # Generazione JWT (omessa)
20         # jwt_token = generate_jwt(...)
21
22         if trigger_status == "PROBLEM":
23             create_ticket(event_id, host_name, jwt_token)
24         elif trigger_status == "OK":
25             resolve_ticket(event_id, jwt_token)
26
27     except Exception as e:
28         logging.error(f"Errore fatale durante l'esecuzione: {e}")
29
30 if __name__ == "__main__":
31     main()

```

Listing 5.2: Estrazione semplificata della gestione parametri e logging nello script Python

Esecuzione in Background, Logging e Troubleshooting

Una delle sfide ingegneristiche maggiori nell'implementazione di Custom AlertScripts in Zabbix è la gestione del contesto di esecuzione. Zabbix esegue lo script in un processo *background* completamente cieco (*headless*), tramite l'utente di sistema `zabbix`. In caso di errore (es. un'eccezione Python, un timeout di rete o un token JWT malformato), l'output non viene mostrato su alcun terminale e il fallimento risulta invisibile (*silent failure*).

Per rendere il sistema affidabile e mantenibile (*troubleshooting*), è stato sviluppato un sistema di tracciamento rigoroso:

1. **Percorsi Assoluti:** Poiché la directory di lavoro corrente (CWD) del processo di Zabbix non è prevedibile, all'interno dello script Python è stato obbligatorio dichiarare esclusivamente percorsi assoluti (es. per il caricamento delle chiavi RSA e per i file di log) per evitare errori bloccanti di `FileNotFound`.
2. **File di Log Dedicato:** Tutte le transazioni HTTP, inclusi i codici di risposta di Interacta (es. HTTP 200 OK, HTTP 500 Server Error) e le eventuali eccezioni, vengono scritte in append nel file `interactaCreateOrSolveTicket_log.log`.
3. **Logrotate:** Dato l'elevato volume di eventi intercettati nella rete Smart City, un file di log non gestito porterebbe rapidamente all'esaurimento dello spazio su disco della macchina virtuale. Per scongiurare questo rischio, è stata definita una direttiva `Logrotate` di sistema che, settimanalmente,

comprime e archivia lo storico, mantenendo un tetto massimo dimensionale (es. 30MB) e applicando correttamente la direttiva su `zabbix zabbix` per non alterare i permessi di scrittura sul file appena ruotato.

Capitolo 6

Analisi Dati, KPI e Manutenzione di Sistema

6.1 Esportazione degli eventi in formato strutturato (CSV)

L'implementazione di un'architettura di monitoraggio centralizzata risolve il problema della rilevazione in tempo reale, ma genera parallelamente una vasta mole di dati storici. Per trasformare questa raccolta di allarmi in informazioni utili al management aziendale (es. calcolo dell'MTTR, individuazione dei colli di bottiglia e analisi del Flapping), è necessario estrarre e analizzare le serie temporali.

Motivazioni e Scelte Architettureali

Zabbix immagazzina nativamente tutti gli eventi nel proprio database relazionale di produzione (tipicamente PostgreSQL o MySQL). In fase di progettazione, si è valutata la possibilità di far collegare il nostro motore di analisi Python direttamente al database SQL. Tuttavia, questa strada è stata scartata per due ragioni ingegneristiche fondamentali:

1. **Impatto sulle Performance:** Eseguire query analitiche complesse e pesanti (come raggruppamenti su decine di migliaia di righe) sullo stesso database utilizzato dal motore di monitoraggio in tempo reale avrebbe introdotto un inutile e rischioso *overhead* prestazionale.
2. **Disaccoppiamento (Decoupling):** Esportare i dati in un formato strutturato e agnostico come il CSV (Comma-Separated Values) disaccoppia il sistema di monitoraggio da quello di reportistica. Questo approccio a file piatto (*flat file*) permette di utilizzare in modo estremamente efficiente librerie Python di data science (come Pandas), bypassando la complessità dello schema relazionale nativo di Zabbix e aggirando le logiche di *retention* che il server applica per cancellare i log più vecchi al fine di risparmiare spazio.

Implementazione tramite Custom Media Type

Per ottenere l'esportazione continua e in tempo reale degli allarmi senza pesare sul database, si è deciso di sfruttare il sistema di *Media Types* di Zabbix. Solitamente, un Media Type definisce il canale di consegna di una notifica (es. Email, SMS o Telegram). Nel nostro caso, è stato creato un Media Type personalizzato di tipo **Script**.

A questo Media Type è stata agganciata un'*Action* configurata per scattare ogni volta che si verifica un problema critico sulla rete Smart City. Invece di inviare una mail, Zabbix esegue lo script `Bash ScriptCustomExportProblemsToCSV.sh` depositato sulla macchina virtuale Ubuntu.

Zabbix passa allo script una stringa pre-formattata contenente tutte le **Macro** necessarie per l'analisi. I parametri in ingresso includono tipicamente:

- `{EVENT.ID}`: L'identificativo univoco dell'allarme.
- `{EVENT.DATE}` e `{EVENT.TIME}`: Timestamp di inizio del disservizio.
- `{EVENT.RECOVERY.DATE}` e `{EVENT.RECOVERY.TIME}`: Timestamp di risoluzione (fondamentale per il calcolo dell'MTTR).
- `{HOST.NAME}` e `{INVENTORY.LOCATION}`: Riferimenti al dispositivo e alla sua ubicazione fisica (es. ID Palo).
- `{TRIGGER.NAME}` e `{TRIGGER.SEVERITY}`: La tipologia del problema e la sua criticità.

Logica di Scrittura del CSV

Lo script Bash sulla VM Linux riceve l'intera stringa di parametri (generalmente racchiusa nel parametro `$1`) e provvede ad accodarla al file CSV di destinazione. La logica dietro questo script è volutamente essenziale: si limita a garantire che la sintassi CSV non venga "rotta" da eventuali virgole o doppi apici presenti nei nomi degli host o dei trigger, utilizzando tecniche di *quoting* appropriate.

```

1 #!/bin/bash
2 # Percorso assoluto del file CSV di destinazione
3 CSV_FILE="/home/administrator/zabbixHostsScript/export_problems.
  csv"
4
5 # Ricezione della stringa formattata da Zabbix
6 INCOMING_DATA="$1"
7
8 # Accodamento (append) al file CSV
9 # L'utilizzo di >> garantisce che il file non venga sovrascritto
10 echo "$INCOMING_DATA" >> "$CSV_FILE"
11
12 # L'uscita a zero conferma a Zabbix il successo dell'operazione
13 exit 0

```

Listing 6.1: Snippet concettuale dello script di esportazione CSV (`ScriptCustomExportProblemsToCSV.sh`)

Grazie a questo flusso, la macchina virtuale Ubuntu viene costantemente alimentata in background. Ogni qualvolta un ticket si apre o si chiude su Interacta, una corrispondente riga di log contenente tutte le variabili temporali ed hardware

viene registrata in sicurezza all'interno di `export_problems.csv`, fornendo un dataset completo, sempre aggiornato e pronto per essere dato in pasto ai successivi script analitici scritti in Python.

6.2 Sviluppo del motore Python per il calcolo dei KPI

Una volta garantita l'esportazione continua degli eventi di allarme su file `system` tramite il Media Type personalizzato e lo script Bash, il passo successivo è l'elaborazione di questa mole di dati grezzi (*Raw Data*). A tale scopo è stato sviluppato un motore di analisi ad-hoc in Python, sfruttando in particolare la libreria `pandas`, lo standard industriale per la manipolazione di dataset tabulari e l'analisi dei dati.

Logica di Elaborazione e Data Preprocessing

Il motore di calcolo si compone di moduli script eseguiti all'interno del *Virtual Environment* della macchina di produzione. Il primo modulo si occupa della fase cruciale di *Data Preprocessing*: il file `export_problems.csv` viene importato in un *DataFrame* Pandas.

Poiché i dati provengono da un output Bash asincrono, è necessario pulire e normalizzare il dataset: si eliminano eventuali righe corrotte, si gestiscono i valori nulli (ad esempio, gli allarmi ancora attivi per i quali non esiste ancora un *Recovery Time*) e si esegue il parsing dei campi data e ora. I campi `{EVENT.DATE}` e `{EVENT.TIME}` vengono concatenati e convertiti in oggetti `datetime` nativi di Python, requisito fondamentale per poter eseguire l'aritmetica temporale necessaria al calcolo dei KPI.

Estrazione dei Key Performance Indicators (KPI)

L'obiettivo primario del motore Python è distillare il dataset per calcolare metriche operative (KPI) essenziali per valutare l'affidabilità dell'infrastruttura Smart City e l'efficienza degli interventi di Secoval. Le logiche algoritmiche si concentrano su tre direttrici principali:

- **Mean Time To Resolution (MTTR):** Questo indicatore misura il tempo medio impiegato dall'azienda per risolvere un guasto a partire dal suo rilevamento. Lo script isola le righe relative ai problemi risolti e calcola la differenza temporale (delta) tra l'istante di ripristino e l'istante di attivazione del trigger. L'MTTR viene poi aggregato per comune o per tipologia di apparato (es. TVCC vs Access Point), offrendo alla direzione tecnica un dato quantitativo per valutare il rispetto degli SLA (*Service Level Agreement*) verso gli enti pubblici.
- **Analisi del Flapping (Instabilità):** Un problema tipico nelle reti geografiche estese, in particolare sui gateway LTE e sugli switch di periferia, è l'alternanza rapida e continua tra lo stato di UP e DOWN (fenomeno del *flapping*). Il motore Python implementa una logica che raggruppa gli

allarmi per `HOST.NAME` e conta la frequenza delle transizioni in finestre temporali ristrette (es. numero di cadute nelle ultime 24 ore). Se un apparato supera una determinata soglia di guardia, viene identificato come "instabile", permettendo di prevenire guasti hardware imminenti o di far intervenire l'operatore telefonico per degrado del segnale.

- **Distribuzione dei Fault e Top-Offenders:** Sfruttando le funzioni di aggregazione (`groupby`) di Pandas, il sistema incrocia i disservizi con i metadati geografici e di inventario importati in fase di discovery. Questo produce una classifica dei *Top-Offenders*, ovvero le reti, gli specifici modelli hardware o i punti geografici (identificati dall'ID Palo) che generano il maggior volume di allarmi. Questa metrica è vitale per guidare gli investimenti di manutenzione straordinaria.

```

1 import pandas as pd
2
3 def calculate_mttr(csv_path):
4     # Importazione del CSV e assegnazione degli header
5     cols = ['EventID', 'Host', 'StartDate', 'StartTime', 'RecDate',
6             'RecTime']
7     df = pd.read_csv(csv_path, sep=';', names=cols)
8
9     # Filtraggio: si considerano solo gli eventi effettivamente
10    risolti
11    df_resolved = df.dropna(subset=['RecDate', 'RecTime']).copy()
12
13    # Conversione delle stringhe in oggetti datetime
14    df_resolved['Start_DT'] = pd.to_datetime(df_resolved['StartDate'] + ' ' + df_resolved['StartTime'])
15    df_resolved['Rec_DT'] = pd.to_datetime(df_resolved['RecDate'] + ' ' + df_resolved['RecTime'])
16
17    # Calcolo del Resolution Time (differenza temporale
18    convertita in ore)
19    df_resolved['Resolution_Time_Hrs'] = (df_resolved['Rec_DT'] -
20    df_resolved['Start_DT']).dt.total_seconds() / 3600
21
22    # Raggruppamento per Host e calcolo della media temporale (
23    MTTR)
24    mttr_by_host = df_resolved.groupby('Host')['Resolution_Time_Hrs'].mean().reset_index()
25
26    return mttr_by_host

```

Listing 6.2: Snippet concettuale in Pandas per il calcolo dell'MTTR per host

Attraverso questo motore analitico, il sistema di monitoraggio cessa di essere un semplice strumento di reazione passiva e diventa uno strumento di analisi predittiva e di misurazione dell'efficienza dei processi aziendali, chiudendo il cerchio dell'automazione progettato durante il tirocinio.

6.3 Automazione reportistica tramite Crontab e SMTP

Il calcolo dei Key Performance Indicators (KPI), descritto nella sezione precedente, genera un valore intrinseco per l'azienda solo se queste informazioni vengono recapitate tempestivamente ai responsabili decisionali. Affinché il sistema passi da un modello reattivo (in cui l'operatore deve interrogare il database) a un modello proattivo, è stato necessario ingegnerizzare un meccanismo di orchestrazione automatica e di distribuzione della reportistica.

Orchestrazione dei Job tramite Crontab

A livello architetturale, l'elaborazione dei log e l'invio delle e-mail sono processi di tipo *batch*. Per automatizzare questi task all'interno della macchina virtuale Ubuntu, ci si è affidati a **Cron**, il demone di sistema standard in ambiente Linux per la schedulazione dei processi.

L'inserimento delle direttive all'interno del file `crontab` dell'utente di sistema ha richiesto un'attenzione particolare alla gestione degli ambienti di esecuzione. Poiché il demone Cron esegue i comandi in una shell non interattiva e con un set di variabili d'ambiente (PATH) ridotto rispetto a un normale login utente, l'uso di comandi generici come `python3 script.py` avrebbe causato il fallimento dell'esecuzione per la mancata risoluzione delle dipendenze (come `pandas` o `requests`).

Per ovviare a questo problema, la direttiva di crontab è stata strutturata richiamando esplicitamente l'interprete Python situato all'interno del *Virtual Environment* (`venv`) del progetto, specificando esclusivamente percorsi assoluti sia per gli eseguibili che per i file di log:

```
1 # Esecuzione automatica ogni giorno alle ore 06:10 del mattino
2 10 6 * * * /home/administrator/zabbixHostsScript/venv/bin/python
   /home/administrator/zabbixHostsScript/generate_daily_report.py
   >> /var/log/zabbix_report_cron.log 2>&1
```

Listing 6.3: Esempio di schedulazione in Crontab per il report mattutino

Questa configurazione garantisce che ogni mattina, prima dell'inizio del turno operativo, il sistema prelevi l'ultimo file `export_problems.csv`, elabori i dati e prepari il report, mantenendo traccia di eventuali errori di esecuzione nel file di log dedicato.

Integrazione SMTP e Distribuzione del Report

Il modulo finale dello script Python si occupa della formattazione e dell'invio dei dati tramite e-mail. Per l'implementazione si è scelto di utilizzare le librerie standard di Python `smtplib` e `email.mime`, le quali permettono di dialogare nativamente con il server di posta interno di Secoval (Mail Relay) senza la necessità di installare agent esterni o tool a riga di comando complessi (come `sendmail` o `postfix`).

Una volta che il motore Pandas ha aggregato i dati (es. MTTR per comune, host instabili per Flapping), i risultati vengono convertiti in tabelle HTML strutturate. La libreria `MIMEMultipart` permette di assemblare l'e-mail combinando un corpo testuale (in formato HTML per una migliore resa visiva) ed eventuali

allegati (es. un file Excel riassuntivo generato on-the-fly dallo script per analisi più approfondite da parte dei manager).

```

1 import smtplib
2 from email.mime.multipart import MIMEMultipart
3 from email.mime.text import MIMEText
4
5 def send_kpi_report(html_content, recipient_email):
6     # Parametri del server SMTP aziendale
7     smtp_server = "smtp.secoval.internal"
8     smtp_port = 25
9     sender_email = "zabbix-reports@secoval.it"
10
11     # Costruzione dell'oggetto MIME
12     msg = MIMEMultipart('alternative')
13     msg['Subject'] = "Report Giornaliero KPI Infrastruttura Smart
14     City"
15     msg['From'] = sender_email
16     msg['To'] = recipient_email
17
18     # Inserimento del contenuto HTML generato da Pandas
19     msg.attach(MIMEText(html_content, 'html'))
20
21     try:
22         # Connessione al mail server e invio
23         with smtplib.SMTP(smtp_server, smtp_port) as server:
24             # server.starttls() # Se richiesto dal server
25             aziendale
26             server.sendmail(sender_email, recipient_email, msg.
27             as_string())
28             print(f"Report inviato con successo a {
29             recipient_email}")
30
31     except smtplib.SMTPException as e:
32         print(f"Errore fatale durante l'invio SMTP: {e}")

```

Listing 6.4: Snippet Python per la composizione e l'invio del report via SMTP

Le motivazioni architetturali di questo approccio risiedono nell'affidabilità e nell'indipendenza del sistema. L'infrastruttura di reportistica vive completamente all'esterno del frontend PHP di Zabbix. Questo significa che, anche in scenari di forte stress del database relazionale, o durante eventuali manutenzioni dell'interfaccia web, il motore Python e Crontab continuano ad accedere ai file testuali locali e a recapitare i report diagnostici, garantendo una *business continuity* totale per il monitoraggio dei livelli di servizio.

6.4 Gestione dei log e manutenzione (Logrotate)

L'implementazione di un'architettura automatizzata e distribuita, come quella sviluppata per Secoval, comporta l'esecuzione silente e continua di molteplici processi in background. Dal middleware Python in ascolto sugli stream ISAPI delle telecamere Hikvision, agli script Bash schedulati tramite Crontab, fino ai *Custom AlertScripts* che dialogano con le API di Interacta, l'intera infrastruttura opera in modalità *headless* (senza interfaccia grafica o terminale interattivo). In questo contesto, un'efficace gestione dei log non è solo una *best practice*, ma

l'unico strumento a disposizione dei sistemisti per garantire l'osservabilità e il *troubleshooting* del sistema in caso di guasti (*silent failures*).

Centralizzazione e Interrogazione dei Log

Ogni componente custom sviluppato durante il tirocinio è stato istruito per generare output di debug strutturati. Ad esempio, il modulo di integrazione con il ticketing registra ogni tentativo di apertura o risoluzione in un file dedicato: `/usr/lib/zabbix/alertscripts/interactaCreateOrSolveTicket_log.log`. A causa dell'imprevedibilità della *Current Working Directory* (CWD) del demone Zabbix, tutti gli script utilizzano rigorosamente percorsi assoluti all'interno del file system Ubuntu.

L'utilizzo di semplici file testuali sequenziali (*flat files*) permette ai tecnici del NOC (Network Operations Center) di Secoval di interrogare il sistema in modo rapido, senza dover accedere a interfacce grafiche complesse o database. Tramite la riga di comando Linux, è possibile:

- Utilizzare `tail -f /percorso/del/log` per monitorare in tempo reale il traffico HTTP in uscita verso Interacta o l'arrivo dei pacchetti XML dalle telecamere.
- Sfruttare `grep` abbinato a espressioni regolari per filtrare e isolare rapidamente tutti i log relativi a uno specifico `{EVENT.ID}` o a un determinato indirizzo IP.

Prevenzione della Saturazione del Disco

Il rovescio della medaglia di un tracciamento così granulare è il consumo di memoria di massa. Una singola telecamera instabile (soggetta a flapping) o un disservizio prolungato delle API cloud possono generare decine di migliaia di righe di log in poche ore. Senza una politica di manutenzione attiva, il disco della macchina virtuale si saturerebbe (Errore *No space left on device*), causando il blocco catastrofico dell'intero motore di monitoraggio Zabbix e del database sottostante.

Per mitigare questo rischio architetturale, l'intero ciclo di vita dei file di log è stato delegato a **Logrotate**, un'utility di sistema nativa in ambiente Linux progettata per ruotare, comprimere e rimuovere i file storici in modo automatizzato.

È stato creato un file di configurazione specifico depositato in `/etc/logrotate.d/zabbix-interacta`. Di seguito è riportato il blocco di codice implementato in produzione:

```
1 /usr/lib/zabbix/alertscripts/interactaCreateOrSolveTicket_log.log
2 {
3     su zabbix zabbix
4     weekly
5     maxsize 30M
6     rotate 12
7     dateext
8     dateformat -%Y-%m-%d
9     olddir /usr/lib/zabbix/alertscripts/storico_log
10    compress
11    delaycompress
12    missingok
13    notifempty
14    create 0600 zabbix zabbix
```

14 }

Listing 6.5: Configurazione logrotate per la manutenzione dei log di integrazione

Logiche Ingegneristiche della Configurazione

Le direttive impiegate in questa configurazione rispondono a specifiche esigenze operative e di sicurezza riscontrate durante la messa in opera:

- **Retention e Compressione:** Le regole `weekly` e `rotate 12` indicano al sistema di conservare lo storico per circa tre mesi (12 settimane). I file vecchi vengono spostati nella cartella `olddir` e compressi tramite l'algoritmo `gzip` (`compress`), riducendo l'ingombro su disco di oltre il 90%. Per evitare saturazioni improvvise, la direttiva `maxsize 30M` forza una rotazione immediata se il file supera i 30 Megabyte, indipendentemente dalla scadenza settimanale.
- **Gestione Permessi (su `zabbix zabbix`):** Questa è una direttiva fondamentale per l'integrità del sistema. Logrotate viene eseguito da Cron con i privilegi massimi di `root`. Se ricreasse il file di log (`create 0600`) assegnandogli la proprietà `root`, lo script Python (eseguito dal processo non privilegiato `zabbix`) si vedrebbe negato l'accesso per la scrittura dei futuri allarmi (Errore *Permission Denied*). La direttiva `su` forza Logrotate a operare nel contesto di sicurezza corretto.
- **Gestione dei Descrittori (Create vs Copytruncate):** Nel caso degli AlertScripts (come quello per Interacta), lo script viene avviato, scrive il log e si chiude. Per questo scenario, la direttiva `create` è ottimale, in quanto il vecchio file viene rinominato e ne viene creato uno nuovo vuoto. Diverso è il caso del demone Python per Hikvision, che è un processo *long-running* con un puntatore al file costantemente aperto in memoria. In quel caso, rinominare il file bloccherebbe il salvataggio dei dati. Pertanto, per i log ISAPI è stata adottata la direttiva `copytruncate`, che copia il contenuto del log e svuota l'originale, permettendo al descrittore del file del processo Python di continuare a funzionare ininterrottamente.

Grazie a questa ingegnerizzazione della reportistica e della manutenzione log, il sistema monitoraggio non si limita a erogare il servizio, ma diviene *self-maintaining*, minimizzando il carico operativo (overhead) per i sistemisti di Secoval. *[Inserimento del codice di configurazione in `/etc/logrotate.d/`].*

Capitolo 7

Conclusioni

7.1 Raggiungimento degli Obiettivi

Il progetto di stage ha perseguito e completato con successo l'obiettivo primario delineato nel patto formativo: la trasformazione dell'infrastruttura di rete periferica (videosorveglianza, access point pubblici e nodi LTE) in un ecosistema Smart City integrato, automatizzato e centralizzato. La migrazione tecnologica dal precedente sistema statico (PRTG) alla piattaforma open-source Zabbix 7.0 ha permesso di superare le criticità legate all'inserimento manuale degli host e alla latenza nella rilevazione dei guasti.

Analizzando i requisiti funzionali stabiliti in fase di analisi, è possibile confermare il pieno raggiungimento dei seguenti traguardi ingegneristici:

- **Automazione dell'Inventario e Integrazione GIS:** L'inserimento dei dispositivi non richiede più operazioni manuali. Lo script Python sviluppato è in grado di interrogare il CMDB aziendale (Excel) e di automatizzare il provisioning tramite le API JSON-RPC di Zabbix. Parallelamente, l'obiettivo di mappare la rete sul territorio è stato raggiunto sviluppando un modulo per l'estrazione delle coordinate GPS dalle relazioni in formato PDF, popolando automaticamente la Geomap del sistema.
- **Monitoraggio Proattivo degli Eventi (L7):** Si è passati con successo da un monitoraggio passivo di livello 3 (ICMP Ping) a un controllo applicativo in tempo reale. Il middleware Python in ascolto sugli stream ISAPI delle telecamere Hikvision garantisce ora l'intercettazione istantanea di eventi critici come il *Video Tampering*.
- **Orchestrazione e Ticketing "Zero-Touch":** L'integrazione con la piattaforma cloud aziendale Interacta è pienamente operativa. L'implementazione di un'architettura Server-to-Server basata su token crittografici JWT (RS512) garantisce l'apertura e la risoluzione automatica e sicura dei ticket, azzerando il carico di lavoro manuale per gli operatori della centrale operativa.

Implementazioni Aggiuntive (Plusvalore del Progetto)

Oltre a soddisfare pienamente i requisiti iniziali, il progetto ha introdotto funzionalità analitiche non previste originariamente, spostando il focus dal semplice "monitoraggio" alla "Business Intelligence" applicata alle infrastrutture, un

aspetto centrale nel percorso di Ingegneria delle Tecnologie per l'Impresa Digitale (ITID).

Nello specifico, il sistema è stato esteso con un motore di calcolo KPI sviluppato in Python (sfruttando la libreria Pandas). Questa implementazione aggiuntiva preleva gli storici degli allarmi (esportati asincronamente in CSV) ed estrae metriche di livello enterprise, quali:

- Il calcolo automatico del **Mean Time To Resolution (MTTR)** per valutare l'efficienza degli interventi sul territorio.
- L'identificazione degli host soggetti a **Flapping**, permettendo al management aziendale di Secoval di attuare politiche di manutenzione predittiva su hardware degradato o su ponti radio LTE instabili.

Infine, l'orchestrazione nativa tramite Crontab e la spedizione automatizzata di questi report via SMTP ha fornito all'azienda uno strumento decisionale proattivo, confermando l'efficacia dell'architettura ingegnerizzata durante il tirocinio.

7.2 Limiti del sistema e Sviluppi Futuri

Sebbene l'architettura implementata abbia raggiunto gli obiettivi prefissati, riducendo drasticamente il carico di lavoro manuale per la gestione dell'infrastruttura Smart City, la messa in produzione ha fatto emergere alcuni limiti intrinseci. Parte di questi limiti derivano da scelte implementative necessarie per rispettare le tempistiche del tirocinio, generando quello che in ingegneria del software viene definito *debito tecnico*.

Un'analisi critica del sistema, documentata anche nelle linee guida interne redatte per i tecnici aziendali, ha permesso di isolare le seguenti aree di miglioramento e di delineare i relativi sviluppi futuri.

Limiti Architettureali e di Sicurezza

- **Hardcoding delle Credenziali:** Attualmente, le password per l'accesso HTTP (Digest Auth) alle telecamere Hikvision e i path delle chiavi crittografiche per la generazione dei JWT verso Interacta sono inseriti in chiaro all'interno degli script Python (*hardcoded*). Questa prassi espone a rischi di sicurezza nel caso in cui un utente non autorizzato ottenga accesso in lettura al file system. *Sviluppo futuro:* Esternalizzare i segreti aziendali implementando un file di configurazione separato (`.env`) letto a runtime tramite la libreria `python-dotenv`, oppure, in un'ottica più *enterprise*, sfruttare nativamente le **Zabbix Vault Macros** (es. integrazione con HashiCorp Vault) per iniettare i segreti in memoria solo al momento dell'esecuzione.
- **Gestione del Codice e degli Ambienti (Dependency Management):** Tutto il codice sorgente (Python, Bash) risiede unicamente all'interno della directory `/home/administrator/zabbixHostsScript`. Manca un controllo di versione formale e un file `requirements.txt` per ricreare l'ambiente virtuale. *Sviluppo futuro:* Inizializzare un repository **Git** locale o remoto per tracciare le modifiche e facilitare il *deployment* del codice. Creare un file `requirements.txt` elencando le dipendenze esatte (es. `pandas`,

`cryptography`, `PyJWT`) per garantire la riproducibilità dell'ambiente in caso di migrazione su un nuovo server Ubuntu.

- **Disaster Recovery e Tolleranza ai Guasti:** Manca una procedura strutturata per far fronte a scenari di errore critico (es. corruzione del database relazionale di Zabbix o prolungata indisponibilità delle API di Interacta con conseguente errore HTTP 500 costante). *Sviluppo futuro:* Implementare logiche di *retry* asincrone con *exponential backoff* nello script di apertura ticket, in modo da non perdere gli allarmi in caso di cloud irraggiungibile. Parallelamente, standardizzare il backup del database Zabbix e gli snapshot della Virtual Machine sull'infrastruttura di virtualizzazione (Nutanix).

Colli di Bottiglia Prestazionali (Performance Bottlenecks)

Dal punto di vista computazionale, il limite maggiore del middleware attuale riguarda l'orchestrazione dei processi secondari. Nello specifico, per inviare i dati a Zabbix o configurare l'ascolto XML delle telecamere, lo script Python si affida pesantemente al modulo `subprocess` per richiamare eseguibili esterni come `zabbix_sender` o script ausiliari in Bash (es. `configura_telecamere.sh`).

In una situazione di regolarità, questo approccio funziona in modo asincrono (*fire-and-forget*) senza causare latenza. Tuttavia, in uno scenario di *Alert Storm* (tempesta di allarmi) – ad esempio se un fulmine causa lo sbalzo di tensione su un intero comune facendo scattare simultaneamente decine di telecamere – il listener Python si troverebbe a lanciare contemporaneamente decine di processi figlio (sub-shell Bash). L'overhead derivante dal continuo *context switching* a livello di sistema operativo comporterebbe un inevitabile picco di utilizzo della CPU della macchina virtuale.

Sviluppo futuro per l'ottimizzazione: Il paradigma deve evolvere da un modello basato sull'invocazione di processi esterni (*multiprocessing* indiretto) a un modello totalmente integrato e asincrono a livello applicativo (*asyncio*). Eliminando gli script Bash di mezzo, le richieste HTTP verso le telecamere e le comunicazioni verso le API possono essere gestite direttamente dal codice Python impiegando la libreria `aiohttp`. L'uso di *coroutine* e di un *event loop* singolo permetterebbe di processare migliaia di eventi concorrenti consumando una frazione della RAM e della CPU attualmente impiegate.

Estensione Orizzontale del Monitoraggio

Dal punto di vista funzionale, avendo validato l'architettura su TVCC e Access Point, lo sviluppo futuro più naturale consiste nell'integrazione di nuove classi di sensori **IoT (Internet of Things)** in via di sviluppo presso i comuni serviti da Secoval. L'infrastruttura Zabbix, ora dotata di script di provisioning API e interfacciata con il ticketing aziendale, è pronta per accogliere nativamente sensori ambientali (qualità dell'aria, PM10), sensori di controllo dei livelli dei fiumi o sistemi di illuminazione intelligente (Smart Lighting), centralizzando ulteriormente il controllo del territorio in un'unica piattaforma ingegnerizzata.

7.3 Valutazione Personale

Il periodo di tirocinio trascorso presso Secoval ha rappresentato una tappa fondamentale nel mio percorso di studi in Ingegneria delle Tecnologie per l'Impresa Digitale, fungendo da ponte cruciale tra l'astrazione della teoria accademica e la complessa concretezza dell'ambiente di produzione aziendale.

Il passaggio dallo studio universitario alla pratica ingegneristica mi ha posto di fronte a sfide che difficilmente si possono simulare in un laboratorio didattico. Mentre all'università lo sforzo si concentra tipicamente sulla correttezza algoritmica o sulla sintassi ideale di un linguaggio, in azienda ho dovuto misurarmi con la natura imprevedibile dei sistemi distribuiti. Ho toccato con mano come l'impostazione errata dei permessi su un file *PEM* o la mancata gestione di un descrittore in ambiente server possano causare *silent failures* bloccanti, rendendo invisibile il disservizio.

Dal punto di vista tecnico, la necessità di orchestrare flussi tra Zabbix, le telecamere Hikvision e la piattaforma cloud Interacta mi ha permesso di espandere e consolidare significativamente le mie competenze nell'ecosistema Linux, nello specifico su architettura Ubuntu. Progettare da zero un'infrastruttura che interseca demoni *systemd*, job *crontab*, script di utilità in *Bash* e middleware avanzati in *Python*, ha trasformato radicalmente la mia concezione della programmazione: il codice non è più inteso come uno script isolato, ma come un ingranaggio robusto all'interno di una catena di automazione profondamente disaccoppiata.

Uno degli apprendimenti più incisivi ha riguardato l'Ingegneria dell'Affidabilità e il *troubleshooting*. Affrontare i malfunzionamenti delle chiamate API HTTP – spesso dovuti non a difetti logici del mio software, ma a latenze di rete o a temporanei errori 500 lato server del provider – mi ha insegnato l'importanza vitale della resilienza architetturale. Ho imparato che un sistema ben ingegnerizzato deve contemplare il fallimento come scenario normale: deve saper gestire le eccezioni, generare log strutturati costantemente sottoposti a *Logrotate* e, se possibile, auto-ripristinarsi.

Infine, l'aver condotto in prima persona l'intero ciclo di vita di questa transizione – partendo dal superamento dei limiti storici del monitoraggio manuale, fino alla definizione di architetture asincrone in grado di abilitare una reportistica avanzata basata sui dati – mi ha fornito una visione olistica dell'IT d'impresa. Oltre all'evidente soddisfazione per aver rilasciato un sistema attualmente in produzione e utile alle squadre tecniche, questa esperienza aziendale mi lascia una solida maturità metodologica. Sento di aver acquisito quel senso pratico e quella capacità di *problem-solving* architetturale che costituiranno le fondamenta indispensabili per affrontare il mio imminente percorso di studi magistrale in ambito informatico. *[Riflessioni su cosa hai imparato: passaggio dalla teoria universitaria alla pratica aziendale, gestione dei problemi, ecc.]*.

Glossario

7.4 Terminologia

Per facilitare la lettura e la comprensione del presente manuale anche a personale non strettamente sistemistico, di seguito vengono definiti i termini tecnici principali, i protocolli e i concetti architetturali utilizzati nel progetto.

Zabbix: è una piattaforma software open-source per il monitoraggio proattivo di infrastrutture IT, reti, server, macchine virtuali e servizi cloud. Consente di raccogliere, visualizzare e analizzare metriche in tempo reale (come utilizzo CPU, RAM, rete e spazio disco), inviando avvisi immediati in caso di anomalie.

Host: qualsiasi dispositivo fisico o virtuale monitorato dal sistema. Nel nostro caso, una telecamera, uno switch, un Access Point o un router.

Item: la singola metrica o il singolo dato raccolto da un Host. Esempi di item sono: la risposta di un comando ping a un host o il tempo di risposta, la risposta ricevuta da un comando, il valore di un sensore fisico o virtuale.

Trigger: una regola logica associata a un Item. Es. la telecamera non risponde al ping per 3 minuti, il Trigger "scatta" e genera un allarme (Problem).

Action: l'azione automatica che il sistema intraprende quando qualche evento soddisfa alcune condizioni. Può essere relativa ai trigger o alla scoperta di un nuovo host.

Template: un modello preconfigurato che contiene un insieme logico di Item, Trigger e regole. Viene applicato a più dispositivi per evitare di configurarli manualmente.

Macro: variabili integrate in Zabbix (scritte tra parentesi graffe, es. {HOST.NAME}) che permettono di inserire dinamicamente il nome, l'IP o altri dati all'interno di svariati campi, per esempio le mail di allarme.

SNMP (Simple Network Management Protocol): il linguaggio standard universale utilizzato dai dispositivi di rete per comunicare il proprio stato di salute e la propria configurazione a un server di monitoraggio centrale.

MIB e OID: Il MIB (Management Information Base) è un dizionario o "mappa" che descrive la struttura dei dati di un dispositivo. L'OID (Object Identifier) è l'indirizzo numerico specifico all'interno di quella mappa (es. 1.3.6.1.4.1.39165.1.1.0) che identifica un singolo valore, come il numero seriale.

API (Application Programming Interface): un'interfaccia o "ponte" software che permette a due sistemi o applicazioni diverse (es. il nostro script Python e Zabbix) di parlarsi, passarsi comandi e scambiarsi dati in modo sicuro e automatizzato.

ISAPI: un set di API specifico e proprietario sviluppato da Hikvision. Permette di interrogare e comandare le telecamere tramite protocollo HTTP e di ricevere flussi continui di avvisi di sicurezza.

GIS (Geographic Information System): sistemi informatici (come QGIS) progettati per acquisire, memorizzare, analizzare e gestire dati georeferenziati. Nel caso di Zabbix permettono di visualizzare gli apparati guasti direttamente su una mappa cartografica.

Comunicazione sincrona: la comunicazione sincrona avviene in tempo reale (es. chiamate, riunioni su Panopto o Zoom, protocollo HTTP). Le decisioni e la gestione dei messaggi devono essere rapide.

Comunicazione asincrona: la comunicazione asincrona non richiede la presenza contemporanea degli interlocutori (es. email, ticket o messaggi su Slack e Dropbox, protocollo MQTT). Consente di gestire i messaggi senza interruzioni e di gestire il proprio tempo.

AP (Access Point): Dispositivo di rete che permette la connessione wireless.

CMDB (Configuration Management Database): Database utilizzato da un'organizzazione per memorizzare informazioni su hardware e software.

Crontab: Demone di sistema nei sistemi operativi Unix-like utilizzato per pianificare l'esecuzione di comandi.

Flapping: Cambiamento frequente e rapido di stato di un host o di una porta di rete, sintomo di instabilità.

GIS (Geographic Information System): Sistema progettato per acquisire, memorizzare, manipolare e analizzare dati spaziali o geografici.

ISAPI (Intelligent Security API): Protocollo basato su HTTP/REST utilizzato dalle telecamere Hikvision per l'integrazione di terze parti.

JWT (JSON Web Token): Standard aperto (RFC 7519) per la trasmissione sicura di informazioni tra due parti come oggetto JSON.

Logrotate: Utility di sistema Linux progettata per amministrare sistemi che generano un gran numero di file di log, ruotandoli e comprimendoli.

MTTR (Mean Time To Resolution): Metrica che indica il tempo medio necessario per riparare un guasto e ripristinare il servizio.

RSA / RS512: Algoritmo di crittografia asimmetrica; RS512 utilizza RSA combinato con la funzione di hash SHA-512 per la firma digitale.

TVCC (TeleVisione a Circuito Chiuso): Sistema di telecamere per la videosorveglianza.

Zabbix Sender: Utility a riga di comando utilizzata per inviare dati di performance o eventi direttamente al server Zabbix in modo asincrono.

ACL (Access Control List): Meccanismo di sicurezza dei file system che definisce permessi granulari di accesso a file e directory per utenti e gruppi specifici.

Bash / Shell: Interprete di comandi testuale predefinito nei sistemi operativi Unix e Linux, utilizzato per interagire con il sistema operativo ed eseguire script.

Batch Processing: Esecuzione di programmi o task in background in modo automatizzato e sequenziale, senza necessità di interazione diretta con l'utente.

Crittografia Asimmetrica: Sistema crittografico che impiega una coppia di chiavi distinte (una pubblica per la verifica/cifatura e una privata per la firma/decifatura) per garantire la sicurezza delle comunicazioni.

Demone (Daemon): Programma eseguito costantemente in background nei sistemi operativi Unix-like, progettato per fornire servizi di sistema o applicativi (come l'ascolto di eventi di rete).

Dependency Hell: Situazione problematica in cui l'installazione o l'esecuzione di un software fallisce a causa di conflitti complessi tra le versioni delle librerie (dipendenze) necessarie.

JSON-RPC: Protocollo di chiamata di procedura remota (RPC) leggero e strutturato che utilizza il formato JSON per codificare le richieste e le risposte tra un client e un server.

JWT (JSON Web Token): Standard aperto (RFC 7519) che definisce un modo compatto e autonomo per trasmettere in modo sicuro informazioni tra le parti come oggetto JSON, validato tramite firma digitale.

Middleware: Componente software che funge da intermediario tra applicazioni, sistemi operativi o protocolli di rete diversi, traducendo e facilitando lo scambio di dati.

Parsing: Processo di analisi sintattica di una sequenza di dati (come un file di testo, XML, JSON o PDF) per estrarne informazioni strutturate e metadati.

Payload e Claim: In un token JWT, il *payload* è il corpo centrale del messaggio. I *claim* sono le singole dichiarazioni o informazioni (es. scadenza, emittente) codificate al suo interno.

Polling: Meccanismo di monitoraggio di tipo attivo in cui un server centrale interroga ciclicamente e a intervalli regolari i dispositivi periferici per ottenerne lo stato operativo.

Regex (Espressioni Regolari): Sequenza di caratteri che definisce un pattern formale di ricerca, ampiamente utilizzata per trovare, validare o manipolare stringhe di testo complesse.

REST (Representational State Transfer): Stile architetturale per lo sviluppo di API nei sistemi distribuiti, basato sull'utilizzo standardizzato dei metodi del protocollo HTTP (GET, POST, PUT, DELETE).

RSA / SHA-512: RSA è uno standard di crittografia asimmetrica; SHA-512 è una funzione di hash crittografico. La loro combinazione (algoritmo RS512) genera firme digitali ad altissima sicurezza.

Symlink (Link Simbolico): File speciale all'interno del file system di un sistema operativo che contiene un riferimento diretto a un altro file o directory, funzionando come un alias.

Trapping: Meccanismo di monitoraggio di tipo passivo, o asincrono, in cui è il dispositivo periferico (o uno script) a inviare proattivamente i dati di allarme al server centrale al verificarsi di un evento.

Trigger / Action: Nella terminologia Zabbix, il *trigger* è l'espressione logica che valuta un parametro per stabilire se si è in uno stato di allarme; l'*action* è l'operazione reattiva automatica scatenata (es. invio mail, avvio script).

Venv (Virtual Environment): Modulo nativo di Python utilizzato per creare ambienti di esecuzione isolati, garantendo che i pacchetti installati per un progetto non interferiscano con le librerie di sistema.

Zabbix Sender: Utility a riga di comando che permette di inviare valori e parametri prestazionali direttamente all'istanza Zabbix Server in modalità asincrona e tramite connessione TCP.

CMDB (Configuration Management Database): Database utilizzato dalle organizzazioni IT per archiviare e gestire in modo centralizzato le informazioni sui Configuration Item (CI), ovvero gli asset hardware, software e le loro relazioni all'interno dell'infrastruttura aziendale.

Data Cleansing (Pulizia dei Dati): Processo informatico di identificazione e correzione (o rimozione) di dati corrotti, inaccurati o non formattati correttamente all'interno di un dataset, fondamentale prima dell'importazione in sistemi di produzione.

ETL (Extract, Transform, Load): Processo di integrazione dei dati che prevede l'estrazione da una o più sorgenti (es. file Excel), la loro trasformazione (normalizzazione e pulizia) e il successivo caricamento in un sistema di destinazione (es. server Zabbix).

Idempotenza: Proprietà di alcune operazioni informatiche (o chiamate API) che garantisce che l'esecuzione ripetuta della stessa operazione produca il medesimo risultato della prima esecuzione, senza alterare ulteriormente lo stato del sistema o creare duplicati.

Infrastructure as Code (IaC): Pratica di gestione e provisioning dell'infrastruttura informatica attraverso file di definizione o script eseguibili in modo automatico, sostituendo le configurazioni manuali e interattive sui sistemi operativi o sui pannelli web.

JSON-RPC: Protocollo di Remote Procedure Call (RPC) leggero e senza stato, che utilizza il formato JSON (JavaScript Object Notation) per codificare le chiamate tra client e server. È lo standard utilizzato nativamente dalle API di Zabbix.

RBAC (Role-Based Access Control): Metodo di regolamentazione dell'accesso a reti e sistemi informatici basato sui ruoli assegnati ai singoli utenti all'interno di un'organizzazione, limitando i privilegi allo stretto necessario (Principio del Minimo Privilegio).

Single Source of Truth (SSoT): Principio di architettura dei sistemi informativi in cui ogni elemento di dato è archiviato e gestito in un unico luogo primario. Qualsiasi altra applicazione che necessita di quel dato lo interroga dalla sorgente autoritativa, evitando disallineamenti.

Virtual Environment (venv): Strumento nativo di Python utilizzato per creare ambienti di esecuzione isolati. Permette di installare librerie e dipendenze specifiche per un singolo progetto senza interferire con l'installazione globale del sistema operativo.

WGS 84 (World Geodetic System 1984): Sistema di coordinate geografiche standardizzato a livello globale, utilizzato come base di riferimento per il sistema di posizionamento satellitare GPS e per le applicazioni di mappatura digitale (GIS).

Polling:

Chunked Transfer Encoding: Meccanismo di trasferimento dati del protocollo HTTP che consente a un server di inviare informazioni in modo continuo (stream) dividendo il payload in una serie di blocchi o frammenti (chunk).

Event-Driven Architecture: Paradigma architetturale del software in cui il flusso del programma è determinato dal verificarsi di eventi esterni (come l'output di un sensore o l'azione di un utente) piuttosto che da un'esecuzione lineare.

Fire-and-forget: Pattern di comunicazione asincrona in cui il mittente invia un messaggio o lancia un processo senza attendere alcun acknowledgement (conferma di ricezione) o risposta da parte del destinatario.

I/O Non Bloccante (Asincrono): Modalità di gestione delle operazioni di input/output in cui un programma, o un processo, può avviare un task e continuare immediatamente la propria esecuzione senza rimanere in attesa del completamento del task stesso.

Isteresi (Hysteresis): In ambito di monitoraggio, tecnica logica che prevede l'utilizzo di soglie differenti per l'attivazione e la disattivazione di un allarme, utile a prevenire ripetute e inutili transizioni di stato (Flapping) in presenza di metriche instabili.

Macro (Zabbix): Variabili dinamiche di sistema (*built-in* o *user-defined*) impiegate in Zabbix per inserire riferimenti a entità logiche, eventi, trigger o proprietà degli host, che vengono risolte a runtime con i valori reali.

Processo Figlio (Subprocess): Nei sistemi operativi, un processo informatico creato e avviato da un altro processo preesistente, denominato "padre". Viene spesso utilizzato per delegare l'esecuzione di specifici comandi esterni.

Push / Pull: Paradigmi architetturali di trasferimento dati sulla rete. Nel modello *Pull* (es. Polling), il client interroga ciclicamente il server per ottenere i dati. Nel modello *Push* (es. Trapping), il nodo sorgente invia i dati proattivamente appena si verifica l'evento.

XML (eXtensible Markup Language): Linguaggio di marcatura flessibile e standardizzato, basato su tag testuali, ampiamente utilizzato per definire strutture gerarchiche di dati e per lo scambio di informazioni tra sistemi eterogenei.

Base64URL: Variante della codifica Base64, progettata specificamente per essere "URL-safe" e sicura per i nomi dei file, sostituendo i caratteri '+' e '/' (problematici nelle URL) con '-' e '_', e rimuovendo i caratteri di padding '='.

CWD (Current Working Directory): In un sistema operativo, è la directory (o cartella) in cui un processo o un utente sta attualmente operando. Gli script eseguiti in background spesso hanno una CWD imprevedibile, rendendo necessario l'uso di percorsi assoluti.

Headless (Esecuzione): Termine utilizzato per descrivere software, processi o script eseguiti in sistemi privi di interfaccia utente grafica (GUI) o di un terminale interattivo connesso, rendendo invisibile l'output standard (stdout).

M2M (Machine-to-Machine): Paradigma di comunicazione in cui dispositivi hardware, sensori o server scambiano informazioni e avviano azioni in modo autonomo e bidirezionale, senza alcun intervento manuale umano.

Man-in-the-Middle (MitM): Tipologia di attacco informatico in cui un'entità malevola intercetta, e potenzialmente altera, la comunicazione segreta tra due parti (es. tra il server Zabbix e il cloud Interacta) all'insaputa di queste ultime.

Non-ripudio (Non-repudiation): Concetto fondamentale della sicurezza informatica (e crittografia) che garantisce che il mittente di un messaggio non possa negare in un secondo momento di averlo inviato, solitamente ottenuto tramite l'uso di firme digitali asimmetriche.

PEM (Privacy Enhanced Mail): Formato di file standard ampiamente utilizzato per memorizzare chiavi crittografiche e certificati digitali (es. chiavi RSA private). Si presenta come file di testo codificato in Base64 e racchiuso tra delimitatori come '—BEGIN PRIVATE KEY—'.

Replay Attack: Attacco di rete in cui una trasmissione di dati valida e sicura (come un token di autenticazione) viene intercettata fraudolentemente o registrata, per poi essere ritrasmessa dall'attaccante per ottenere accessi non autorizzati.

Silent Failure: Fallimento di un programma o di un sistema che si verifica senza generare messaggi di errore evidenti, notifiche o log, rendendo il *troubleshooting* e la diagnosi estremamente complessi per l'amministratore di sistema.

Zero-Touch (Provisioning/Automation): Metodologia di gestione dei sistemi IT in cui i dispositivi o le procedure (come l'apertura di un ticket) vengono configurati ed eseguiti automaticamente dal sistema centrale, riducendo a zero la necessità di inserimento dati manuale da parte di un operatore.

CSV (Comma-Separated Values): Formato testuale aperto (RFC 4180) utilizzato per l'importazione ed esportazione di dati in forma tabellare. Ogni riga rappresenta un record e i campi sono separati da un delimitatore (spesso la virgola o il punto e virgola).

DataFrame: Struttura dati bidimensionale, mutabile e potenzialmente eterogenea, fornita dalla libreria Python **pandas**. Rappresenta il formato standard per la manipolazione, il filtraggio e l'analisi di dati tabulari.

Decoupling (Disaccoppiamento): Principio di ingegneria del software e dei sistemi per cui due o più componenti vengono progettati per operare in modo indipendente l'uno dall'altro. Rende il sistema più resiliente e facile da mantenere.

Flat File: Un database a "file piatto", ovvero un file di testo semplice (come un log o un CSV) che memorizza i dati senza relazioni strutturali complesse, a differenza dei database relazionali (SQL).

ITIL (Information Technology Infrastructure Library): Insieme di best practice internazionalmente riconosciute per la gestione dei servizi IT (IT Service Management), focalizzato sull'allineamento dei servizi informatici con le esigenze di business aziendali.

KPI (Key Performance Indicator): Indicatore chiave di prestazione. È una metrica quantificabile utilizzata per valutare l'efficienza e l'efficacia con cui un'azienda (o un sistema IT) sta raggiungendo i propri obiettivi operativi strategici.

MIME (Multipurpose Internet Mail Extensions): Standard internet (RFC 2045) che estende il formato dei messaggi di posta elettronica per supportare testi con codifiche diverse dall'ASCII, allegati (file, immagini) e contenuti multi-parte (es. email in HTML).

MTTR (Mean Time To Resolution): Tempo medio di risoluzione. Metrica fondamentale nell'Incident Management che calcola il tempo medio impiegato per riparare un guasto e ripristinare la piena funzionalità di un servizio.

NOC (Network Operations Center): Struttura centralizzata da cui gli amministratori di rete supervisionano, monitorano e mantengono operativa l'infrastruttura di telecomunicazioni di un'organizzazione (es. i tecnici Seccoval).

Overhead: In informatica, indica le risorse accessorie di calcolo (tempo di CPU, memoria, banda di rete) richieste per compiere un'operazione, che non contribuiscono direttamente al risultato finale dell'applicazione.

SLA (Service Level Agreement): Accordo sul Livello di Servizio. Contratto (o patto interno) tra un fornitore di servizi IT e i propri clienti che definisce formalmente il livello di prestazione atteso (es. uptime del 99.9%, MTTR inferiore a 4 ore).

SMTP (Simple Mail Transfer Protocol): Protocollo standard di Internet (RFC 5321) impiegato per la trasmissione e l'instradamento di messaggi di posta elettronica tra server.

Troubleshooting: Processo logico e sistematico di ricerca, identificazione e risoluzione delle cause profonde di un problema all'interno di un sistema informatico o di una rete.

Asyncio / Coroutine: Modulo nativo di Python per la scrittura di codice concorrente e asincrono single-thread. Le *coroutine* sono speciali funzioni che possono mettere in pausa la propria esecuzione (ad esempio in attesa di una risposta di rete) per restituire il controllo al ciclo principale (*event loop*), senza bloccare il programma.

Context Switching: Operazione computazionalmente dispendiosa attraverso la quale il sistema operativo salva lo stato del processo o thread attualmente in esecuzione per caricare e avviare quello di un altro processo (come nel caso dell'avvio continuo di script Bash tramite subprocess).

Debito Tecnico (Technical Debt): Metafora dello sviluppo software che indica il costo implicito e il lavoro extra che si renderà necessario in futuro a causa di scelte implementative rapide o provvisorie adottate nel presente per rispettare scadenze stringenti.

Disaster Recovery: Insieme di policy, procedure e strumenti tecnologici atti a garantire il ripristino dell'infrastruttura IT e dei dati vitali di un'azienda a seguito di eventi catastrofici, guasti hardware critici o attacchi informatici.

Exponential Backoff: Algoritmo di gestione degli errori di rete in base al quale un client, a seguito di una richiesta fallita (es. server irraggiungibile), riprova a connettersi aumentando il tempo di attesa in modo esponenziale tra un tentativo e l'altro, per non sovraccaricare ulteriormente il server in difficoltà.

Git / Version Control System (VCS): Sistema software progettato per tenere traccia delle modifiche apportate ai file (in particolare codice sorgente) nel tempo, permettendo di ripristinare versioni precedenti, collaborare in team e gestire le release in modo strutturato.

Hardcoding (Codifica fissa): Pratica di sviluppo, generalmente considerata una cattiva abitudine (anti-pattern), che consiste nell'inserire dati variabili (come password, percorsi di file o indirizzi IP) direttamente nel codice sorgente anziché ricavarli da file di configurazione esterni.

HashiCorp Vault: Strumento software enterprise per la gestione sicura, la conservazione e il controllo degli accessi a segreti, certificati, password e token crittografici utilizzati all'interno di infrastrutture IT moderne.

IoT (Internet of Things): Rete di oggetti fisici ("cose") dotati di sensori, software e altre tecnologie allo scopo di connettersi e scambiare dati con altri dispositivi e sistemi su Internet (es. sensori ambientali Smart City).

Resilienza (Resilience): In ingegneria informatica, la capacità di un sistema, un'architettura o una rete di resistere a condizioni anomale, assorbire guasti hardware o software imprevisti e continuare a fornire un livello di servizio accettabile senza interruzioni totali.

Site Reliability Engineering (SRE): Disciplina ingegneristica introdotta da Google che applica i principi dello sviluppo software alla gestione delle operazioni IT, con l'obiettivo di creare sistemi software ultra-scalabili, stabili e altamente affidabili.

Bibliografia

- [1] Zabbix LLC, *Zabbix 7.0 Manual*. [Online]. Disponibile su: <https://www.zabbix.com/documentation/>
- [2] Python Software Foundation, *Python Programming Language*. [Online]. Disponibile su: <https://www.python.org/>

Bibliografia

Secoval S.r.l. (2026). *Chi Siamo*. Secoval. Recuperato giugno 19, 2026, da https://www.secoval.it/pagina21048_chi-siamo.html

Ringraziamenti

[Spazio dedicato ai ringraziamenti: al Prof. Gringoli, al tutor aziendale in Secoval, famiglia e amici].